Contents

1 INTRODUCTION

1.1 Relevance and problem statement

Temperature control is a common engineering task in laboratory equipment, heating units, environmental chambers, embedded automation systems, and industrial processes. In such systems, the temperature value must not only be measured, but also maintained near a specified setpoint under the influence of disturbances, actuator limitations, sensor errors, thermal inertia, and communication delays. For this reason, a temperature-control system has to be considered not only as a controller, but as an engineering workflow that includes measurement, control action, supervision, configuration, feedback, and verification [1].

A common limitation of simple temperature-control prototypes is that they demonstrate either the local control algorithm or the monitoring interface, but do not always implement the complete engineering loop around the controlled object. In a practical system, the value of control is not determined only by the ability to calculate an actuator signal. It also depends on whether telemetry is transmitted in a structured form, whether commands are applied in a controlled and traceable way, whether parameter changes are acknowledged by the device, whether historical behavior is stored for analysis, and whether the operator can verify the result after a configuration change.

The diploma project is focused on the development of a layered closed-loop temperature control and monitoring system. The system includes an edge control layer, a Data Hub layer, an HMI layer, and an auxiliary decision-support mechanism. The edge layer performs local temperature acquisition and control execution. The Data Hub receives telemetry, parses control-related messages, stores historical data, tracks device state, and prepares information for higher-level processing. The HMI provides operator supervision, telemetry viewing, and parameter configuration. The decision-support mechanism helps analyze control behavior and prepare parameter recommendations, but it is treated as an auxiliary part of the system, not as the main subject of the work. MQTT-based exchange is used because it provides a lightweight publish-subscribe communication model suitable for device-to-service message flow [2].

The relevance of the project is determined by the need to connect embedded control, structured data exchange, persistent storage, operator interaction, and post-apply result checking into one reproducible engineering process. Therefore, the project is not limited to creating a temperature display, a separate PID controller, or an isolated web dashboard. Its purpose is to implement a complete control cycle: temperature measurement, local control, telemetry publishing, Data Hub ingestion, storage and

processing, HMI monitoring, parameter update, device acknowledgement, and verification after the applied change.

The main engineering contribution of the project is the implementation of an end-to-end closed-loop platform in which local control, MQTT-based message exchange, Data Hub processing, persistent historical storage, HMI-based operation, parameter acknowledgement, and post-apply verification are integrated into one reproducible workflow. The auxiliary decision-support mechanism increases the engineering value of the system by supporting analysis and parameter preparation, while the operator remains in the loop and the system emphasizes explainability, feedback, and controlled evaluation rather than uncontrolled automatic optimization.

1.2 Aim and objectives of the project

The aim of the diploma project is to design, implement, and validate a layered closed-loop temperature control and monitoring system that combines an edge control layer, a Data Hub layer, an HMI layer, and an auxiliary decision-support mechanism for control-behavior analysis and parameter recommendation.

To achieve this aim, the following objectives must be completed:

- to analyze temperature control principles, monitoring requirements, and layered closed-loop architecture for an edge-based control system;
- to define the functional and non-functional requirements for the developed system;
- to design the architecture of the edge control layer, the Data Hub layer, and the HMI layer;
- to implement edge temperature acquisition, local control, runtime parameter handling, telemetry publishing, and command acknowledgement;
- to implement Data Hub MQTT ingestion, message parsing, telemetry filtering, aggregation, persistent storage, and device status tracking;
- to implement HMI monitoring, telemetry viewing, parameter configuration, and observation of control-related events;
- to include an auxiliary decision-support mechanism for analyzing control behavior and preparing parameter recommendations;
- to validate the complete closed loop, including telemetry generation, Data Hub ingestion, HMI monitoring, parameter update, device acknowledgement, and observation of system behavior after the applied change.

1.3 Object and subject of the project

The object of the project is an edge-based temperature control system considered as a closed-loop engineering platform. The platform combines local control execution with supervisory data processing, operator interaction, parameter adjustment, and evidence-based result checking.

The subject of the project is the architecture, communication flow, control logic, data processing pipeline, HMI interaction, parameter update mechanism, acknowledgement procedure, verification process, and auxiliary decision-support functions required to implement the complete temperature-control loop.

1.4 Practical significance

The practical significance of the project lies in the implementation of a complete layered prototype that demonstrates an industrial-style workflow for temperature control and monitoring. The system is not limited to displaying temperature values. It supports local control execution, structured MQTT communication, telemetry ingestion, time-series storage, operator interaction, parameter update, acknowledgement handling, and checking of the result after a new configuration is applied.

The edge layer provides a reproducible control environment based on an ESP32-oriented firmware structure and a simulation profile. This makes it possible to test control behavior before transferring the approach to physical hardware. The separation of local control from supervisory processing is important because time-critical actions remain close to the controlled object, while telemetry storage, analysis, visualization, and configuration are handled by higher software layers.

The Data Hub layer improves traceability by receiving telemetry, processing messages, storing historical values, and tracking device state. This allows the operator and the system developer to observe not only the current temperature, but also the sequence of events around parameter changes, acknowledgements, and control response. The HMI layer provides a practical interface for monitoring the system and applying configuration changes in a controlled way.

The auxiliary decision-support mechanism adds value by helping evaluate control behavior and prepare parameter recommendations. At the same time, the operator remains responsible for applying changes through the HMI and checking the result using stored measurements and acknowledgement information. This makes the prototype suitable as an engineering demonstration of a closed-loop control platform with traceable configuration, feedback, and verification. The developed system can also serve as a

foundation for future hardware deployment, extension of control functions, or integration with additional monitoring tools.

1.5 Structure of the thesis

The thesis consists of seven main sections.

The first section introduces the relevance of the topic, states the engineering problem, defines the aim and objectives of the project, and describes the object, subject, and practical significance of the developed system.

The second section analyzes temperature control and monitoring systems. It considers feedback control principles, edge-based execution, telemetry exchange, HMI requirements, decision-support functions, and the requirements for the developed system.

The third section describes the system architecture. It explains the responsibilities of the edge control layer, the Data Hub layer, and the HMI layer, and presents the closed-loop flow of measurements, commands, acknowledgements, and feedback.

The fourth section describes the implementation of the edge control layer, including temperature acquisition, heater control, the control algorithm, runtime configuration, MQTT communication, and local response to parameter updates.

The fifth section describes the implementation of the Data Hub and HMI layers. It covers MQTT ingestion, message parsing, telemetry processing, persistent storage, backend services, frontend interaction, monitoring, and parameter configuration.

The sixth section describes the auxiliary decision-support mechanism and system validation. It explains how telemetry is used for behavior analysis, how recommendations are prepared, and how the complete loop is checked through telemetry generation, data ingestion, HMI operation, acknowledgement, and post-apply observation.

The seventh section summarizes the completed work, evaluates whether the project objectives have been achieved, and outlines possible directions for further improvement.

Thus, the thesis follows the engineering flow of the project: requirements analysis, architecture design, implementation of the main system layers, validation of the closed loop, and final evaluation of the obtained result.

2 ANALYSIS OF TEMPERATURE CONTROL AND MONITORING SYSTEMS

2.1 Temperature-control process characteristics

Temperature control is a common engineering task in heating units, laboratory equipment, environmental chambers, embedded automation systems, and industrial processes. The controlled variable is temperature, while the desired operating condition is defined by a setpoint. The purpose of the control system is to keep the measured temperature close to this setpoint despite internal process dynamics and external influences [1].

A temperature-control process has several characteristics that make it more complex than a simple measurement task. One of the main characteristics is thermal inertia. When the actuator output changes, the temperature of the controlled object does not change immediately. Heat is accumulated in the object, transferred through its material, and dissipated to the surrounding environment. As a result, the measured response appears with delay, and the controller must take into account that the effect of the current actuator signal will become visible only after some time.

Another important characteristic is the influence of disturbances. Ambient temperature changes, airflow, load variation, heat losses, sensor placement, and power-supply instability can affect the process. These disturbances may shift the temperature away from the setpoint even if the controller settings remain unchanged. Therefore, the system must be able to react to changing conditions and not rely only on a fixed actuator output.

Sensor noise and measurement uncertainty also affect temperature-control quality. A sensor may produce small fluctuations even when the physical temperature is nearly constant. If the controller reacts too strongly to such fluctuations, unnecessary actuator changes and oscillations may appear. At the same time, excessive filtering can make the control response slower. This creates a practical trade-off between measurement stability and control responsiveness.

Actuator limitations must also be considered. A heater, fan, relay, valve, or PWM-controlled output has a limited operating range and cannot produce an unlimited control effect. If the actuator reaches saturation, the system may not be able to reduce the control error quickly. Incorrect controller settings may lead to overshoot, when the temperature exceeds the setpoint, or to steady-state error, when the temperature remains below or above the required value for a long time. These characteristics show that temperature control requires not only measurement, but also stable control logic, parameter management, and observation of process behavior.

2.2 Closed-loop control principles

A closed-loop control system uses feedback from the controlled object to correct its own behavior. The basic control sequence includes measurement of the process value, comparison with the setpoint, calculation of the control action, application of this action to the actuator, and repeated observation of the result. This repeated correction allows the system to compensate for disturbances and process changes that cannot be fully predicted in advance.

In a temperature-control system, the measured temperature is periodically read from a sensor and compared with the required setpoint. The difference between these values is used to calculate the actuator output. If the temperature is lower than the setpoint, the controller may increase heating power. If the temperature is higher than required, the controller may reduce the output or switch the actuator off. The process is repeated continuously, so each new measurement becomes feedback for the next control decision.

Closed-loop control is more suitable for temperature regulation than open-loop control because the actual process response can change over time. The same actuator output may produce different temperature behavior depending on environmental conditions, thermal load, heat losses, and the current state of the object. Without feedback, the system cannot determine whether the expected result has been achieved. With feedback, the controller can adjust its output according to the measured response.

In the developed project, the closed-loop principle is extended beyond the local controller. The local loop remains responsible for measurement, control calculation, and actuator output. However, the complete engineering workflow also includes supervisory parameter changes and result verification. A parameter update is prepared in the HMI, published as an MQTT command, received by the edge device, acknowledged after processing, stored by the Data Hub, and then checked through subsequent process behavior. This extended loop is important because it connects operator action with device response and post-apply verification.

2.3 Edge-based execution in temperature-control systems

Edge-based execution means that the time-critical part of the control system is performed close to the controlled object. In a temperature-control system, this approach is important because measurement, control calculation, and actuator output must continue even if the HMI, database, or network connection is temporarily unavailable. The local controller must not depend on a remote service for every control decision.

The edge control layer is therefore responsible for direct interaction with the controlled process. It acquires temperature values, executes the local control algorithm, applies actuator output, maintains runtime parameters, publishes structured telemetry, receives parameter commands, and returns acknowledgements. These functions must be coordinated so that communication activity does not interrupt the local feedback process.

Keeping control execution at the edge also reduces the influence of communication delay. If every actuator update depended on a remote server, unstable network conditions could lead to delayed or missed control actions. In contrast, edge-based execution allows the device to continue operating with the last valid configuration while higher layers provide monitoring, storage, configuration, and analysis.

For the developed system, the edge control layer is the execution core of the closed-loop temperature control and monitoring system. It is not only a data source for the HMI, but also the component that applies control logic and confirms configuration changes. This role requires clear separation between local control responsibilities and supervisory functions implemented in the Data Hub and HMI layers.

2.4 Data acquisition and monitoring requirements

Data acquisition in a temperature-control system must provide structured and traceable information about the controlled process. The system should not collect only temperature values, because a single value does not explain the state of the controller or the reason for a change in behavior. For engineering analysis, each record should include context that allows the process state to be reconstructed.

The required context includes the device identifier, timestamp, measured temperature, setpoint, actuator output, controller state, command identifier, acknowledgement status, parameter version, and abnormal conditions. These fields make it possible to compare behavior before and after parameter changes, determine which configuration was active, and understand whether the edge device accepted a command.

MQTT is suitable for this type of system because it provides lightweight message exchange between the edge device and supervisory software [2]. Separate topics or message types can be used for structured telemetry, parameter commands, acknowledgements, and system events. This separation improves traceability and makes the communication flow easier to process and verify.

The Data Hub layer is responsible for ingestion and processing of these messages. It subscribes to MQTT topics, parses payloads into stable data models, filters or aggregates measurements when necessary, stores normalized records, and tracks device status. This layer separates raw message exchange from application-level use. The HMI

does not need to consume every edge message directly; instead, it can use processed historical records, summaries, device status, and acknowledgement information.

Persistent storage is required for engineering analysis and post-apply verification. Historical records allow the system developer and operator to observe trends, identify slow response or oscillation, compare control behavior under different parameters, and prepare data for auxiliary decision support. Device status tracking is also necessary because stale data, disconnection, or missing acknowledgements can affect the interpretation of control results.

2.5 HMI requirements for supervisory control

The HMI layer is one of the main layers of the developed system. Its role is to provide supervisory control and operator interaction. The HMI must not be limited to displaying the current temperature. It should provide a clear view of the current device state, historical behavior, parameter configuration, command results, alarms, and abnormal events.

For device monitoring, the HMI should show the measured temperature, setpoint, actuator output, controller state, device status, and relevant operating indicators. This information allows the operator to understand whether the system is stable, whether the temperature is approaching the setpoint, and whether the actuator is operating within expected limits.

Historical behavior viewing is also required. The operator must be able to compare current operation with previous periods and observe how the system responded after configuration changes. Such before-and-after observation is especially important when controller parameters are updated, because the value of a parameter change can be evaluated only by checking the subsequent process response.

The HMI must also support controlled parameter configuration. A parameter update should be applied through a clear operator action, transmitted to the edge device, and followed by visible command-result information. The operator should be able to see whether the command was sent, whether the edge device acknowledged it, and whether the system behavior changed after the new parameter version became active.

Alarm and abnormal event visibility is another important requirement. If the device becomes unavailable, measurements become stale, actuator output reaches an abnormal state, or the process moves outside an acceptable range, the HMI should make this condition visible. In this way, the HMI becomes part of the supervisory control process rather than only a passive dashboard.

2.6 Auxiliary decision-support role

The auxiliary decision-support mechanism is an additional support function in the developed system. It is not the main controller, not an autonomous optimizer, and not the central contribution of the thesis. Its role is to analyze stored behavior and prepare reviewable parameter recommendations that may help improve control quality.

Decision support depends on the data collected by the edge control layer and processed by the Data Hub layer. Historical temperature behavior, setpoint changes, actuator output, parameter versions, acknowledgement records, and post-apply results provide the basis for analyzing how the system reacted under different conditions. Without structured telemetry and persistent storage, such analysis would not be traceable.

The recommendation process must remain connected to operator supervision. A recommendation should be reviewed in the HMI before application. If the operator decides to apply it, the parameter update is sent through the normal command mechanism, acknowledged by the edge device, and checked using subsequent measurements. This means that decision support assists the operator but does not bypass the established control and verification workflow.

This boundary is important for the thesis. The main engineering result is the layered closed-loop temperature control and monitoring system. The auxiliary decision-support mechanism increases the value of the system by improving analysis and parameter preparation, while the operator remains in the loop and the system remains explainable and traceable.

2.7 Requirements for the developed system

The analysis of temperature-control process characteristics, closed-loop control principles, edge-based execution, data acquisition, HMI supervision, and auxiliary decision support leads to the requirements for the developed system. These requirements define the difference between a simple temperature-monitoring prototype and a complete layered closed-loop temperature control and monitoring system.

The requirements are grouped by the main system areas: the edge control layer, communication mechanism, Data Hub layer, HMI layer, auxiliary decision-support mechanism, and integrated system validation. This grouping reflects the architecture of the project and keeps the auxiliary functions separated from the main control and monitoring layers. The summarized requirements are presented in Table 2.1.

Table 2.1 – Main engineering requirements of the developed system

Area	**Requirement**	**Purpose**
Edge control layer	Acquire temperature, execute local control, apply actuator output, maintain runtime parameters, publish telemetry, receive commands, and return acknowledgements	Local reliability and control continuity
Communication mechanism	Separate telemetry, commands, acknowledgements, and events using structured messages	Traceability and predictable data exchange
Data Hub layer	Ingest MQTT messages, parse payloads, store normalized records, and track device status	Data consistency and engineering analysis
HMI layer	Provide monitoring, history viewing, parameter configuration, and command-result visibility	Controlled operator interaction
Auxiliary decision-support mechanism	Analyze historical behavior and prepare reviewable parameter recommendations	Explainability and assisted decision-making
Validation	Test the complete path from measurement to post-apply observation	End-to-end verification

3 ARCHITECTURE OF THE DEVELOPED SYSTEM

3.1 General architecture of the system

The developed system is designed as a layered closed-loop temperature control and monitoring system. Its architecture separates time-critical local control from supervisory data processing and operator interaction. This separation is necessary because the temperature-control process must continue at the edge, while monitoring, storage, parameter configuration, and analysis can be performed by upper layers. Figure 3.1 presents the general architecture of the developed system and shows the relationship between the three main layers and the auxiliary decision-support mechanism.

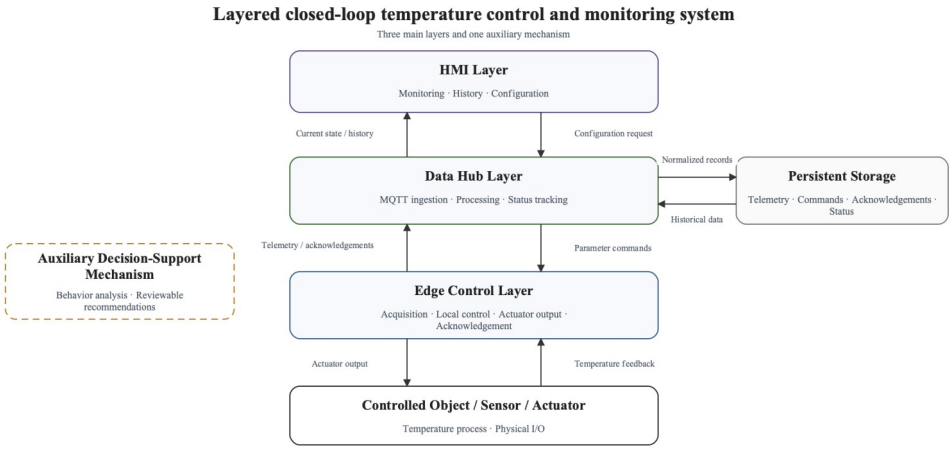

Figure 3.1 – General architecture of the developed system

The architecture contains three main layers: the edge control layer, the Data Hub layer, and the HMI layer. The edge control layer interacts directly with the controlled object and executes the local feedback process. The Data Hub layer receives MQTT messages, parses them, stores normalized records, and maintains device status. The HMI layer provides operator supervision, history viewing, parameter configuration, and command-result visibility. The auxiliary decision-support mechanism is connected to the HMI and stored data, but it is not a separate main control layer and does not replace the operator.

The main architectural decision is to treat the system as an engineering workflow rather than as a set of independent modules. A temperature measurement is not only displayed to the user; it becomes part of a traceable sequence that includes control execution, telemetry publishing, message ingestion, storage, HMI observation, parameter update, device acknowledgement, and post-apply verification. As shown in Figure 3.1, the three main layers are aligned around the closed-loop measurement, command, and verification path, while the auxiliary decision-support mechanism remains outside the

main control chain. The layered structure of the developed system is summarized in Table 3.1.

Table 3.1 – Architectural areas of the developed system

Area	**Main responsibility**	**Main exchanged information**
Edge control layer	Local measurement, control execution, actuator output, runtime parameter handling, telemetry publishing, command receiving, and acknowledgement	Temperature, setpoint, controller parameters, actuator output, device state, acknowledgement data
Data Hub layer	MQTT ingestion, payload parsing, filtering, aggregation, persistent storage, and device-status tracking	Telemetry records, parameter commands, acknowledgements, summaries, status events
HMI layer	Operator monitoring, history viewing, parameter configuration, command-result observation, and abnormal-event visibility	Current state, historical data, parameter forms, command status, alarms
Auxiliary decision-support mechanism	Analysis of stored behavior and preparation of reviewable parameter recommendations	Historical windows, control-performance features, recommendation metadata, post-apply comparison

The three main layers are connected through a common communication and data model. MQTT is used for runtime message exchange between the edge device and upper layers [2], while persistent storage provides the historical basis for monitoring, analysis, and verification. This approach allows each layer to have a clear responsibility while still supporting an end-to-end control workflow.

3.2 Edge control layer

The edge control layer is the execution core of the system. It is responsible for acquiring temperature data, comparing the measured value with the setpoint, calculating

the local control action, and applying the actuator output. In the developed project, the edge node is represented by an ESP32-oriented firmware structure and a Wokwi simulation profile. This makes it possible to test the control workflow in a repeatable environment while keeping the structure suitable for later hardware deployment.

The edge layer must remain operational even when the HMI, database, or network connection is temporarily unavailable. For this reason, the local control algorithm and actuator output are not delegated to the Data Hub or the HMI. The edge device continues to use the last valid runtime configuration and publishes its state when communication is available. This design protects the time-critical part of the control loop from supervisory-layer delays.

Besides local control, the edge layer participates in the wider engineering loop. It publishes structured telemetry with process values and controller state, receives parameter commands, validates or applies updated parameters, and returns acknowledgements. The acknowledgement is important because it distinguishes a command that was only sent from a command that was actually processed by the device.

The telemetry produced by the edge layer includes not only the current temperature but also engineering context such as setpoint, control output, controller coefficients, timing information, safety state, connectivity state, and pending-parameter status. These fields allow the upper layers to interpret process behavior and to verify changes after new parameters are applied.

3.3 Data Hub layer

The Data Hub layer is the processing center between the edge device and the HMI. Its role is to receive MQTT messages, classify them by topic and payload type, transform them into stable internal records, and store them for later use. This layer prevents the HMI from depending directly on raw device messages and provides a consistent data source for monitoring, history, alarms, and analysis.

In the developed system, the Data Hub subscribes to telemetry, parameter-command, acknowledgement, and configuration-update topics. The ingestion pipeline must handle messages in a controlled way, because data can arrive faster than it can be written to storage or displayed to the user. Therefore, the layer includes bounded processing and backpressure concepts. This is important for engineering reliability because uncontrolled buffering can hide overload until the system becomes unstable.

Persistent storage is a central responsibility of the Data Hub. Telemetry records, parameter commands, acknowledgements, summaries, and device-status events are stored so that system behavior can be reconstructed later. This is required for traceability and for post-apply verification. If the operator changes controller parameters, the stored records

show which command was sent, whether the device acknowledged it, what parameter version became active, and how the temperature behavior changed afterwards.

The Data Hub also supports device-status tracking. Online or offline state, stale telemetry, missing acknowledgements, and abnormal process conditions affect how the operator should interpret the HMI view. By maintaining status information separately from raw telemetry, the system can present a clearer operational picture.

3.4 HMI layer

The HMI layer is the operator-facing part of the system. Its role is not limited to displaying a temperature value. It provides access to current device state, historical behavior, parameter configuration, command status, alarms, and post-apply observations. The HMI is therefore the supervisory control layer through which the operator interacts with the developed system.

For monitoring, the HMI displays current temperature, setpoint, actuator output, controller state, connectivity state, and relevant abnormal conditions. For historical analysis, it allows the operator to inspect behavior over time and compare operation before and after parameter changes. This is necessary because the effect of a temperature-control parameter update cannot be evaluated from a single measurement.

The HMI also provides the controlled path for parameter configuration. When the operator applies a new setpoint or controller parameter set, the HMI initiates a parameter command through the system command path. The corresponding upper-layer service publishes the command to the device-specific MQTT topic. The result must not be treated as successful only because the message was published. The HMI should show whether the edge device returned an acknowledgement and whether the subsequent measurements confirm the expected behavior.

This design keeps the operator in the loop. Even when an auxiliary recommendation is available, the HMI remains the place where the recommendation is reviewed, applied, and checked. This boundary is important because the system is intended as a traceable engineering prototype rather than an autonomous optimization product.

3.5 Closed-loop data and command flow

The central system behavior is the closed-loop flow from measurement to post-apply verification. At the local level, the edge device measures temperature, calculates the control action, applies actuator output, and repeats this process. At the system level,

the same control process is extended by structured communication, storage, HMI supervision, and controlled parameter updates.

The normal telemetry path begins when the edge device publishes a message containing the measured temperature, setpoint, control output, controller state, and related status fields. The Data Hub ingests this message, parses it, stores a normalized record, and updates status information. The HMI then uses the stored and processed data to display current and historical behavior to the operator.

The command path begins when an operator changes a parameter through the HMI. The HMI initiates a parameter-set operation, and the upper-layer command path publishes the corresponding message to the device-specific MQTT topic. The edge device receives the command, validates its content, applies or stages the new parameters according to the command semantics, and publishes an acknowledgement. The Data Hub stores the command and acknowledgement records, while the HMI displays the result to the operator.

Post-apply verification closes the engineering loop. After the acknowledgement is received, later telemetry is observed to determine whether the process behavior changed as expected. This step is essential because an acknowledgement only confirms that the device processed the command; it does not prove that the control behavior improved. The sequence of the developed workflow is summarized in Table 3.2.

Table 3.2 – Closed-loop workflow of the developed system

Step	**Responsible layer**	**Purpose**
Temperature measurement and local control	Edge control layer	Maintain local feedback control near the controlled object
Telemetry publishing	Edge control layer	Send structured process and controller state to upper layers
Message ingestion and storage	Data Hub layer	Preserve telemetry and status records for monitoring and analysis
Operator monitoring	HMI layer	Display current and historical behavior in a supervisory interface
Parameter update	HMI layer	Send controlled configuration changes through the command path

Step	Responsible layer	Purpose
Device acknowledgement	Edge control layer and Data Hub layer	Confirm whether the command was processed and store the result
Post-apply verification	Data Hub layer and HMI layer	Compare subsequent behavior with the previous operating state

This flow shows that the developed architecture is not a simple one-way monitoring chain. It contains feedback at the control level and traceability at the engineering-system level.

3.6 Communication protocol and data model

MQTT is used as the main runtime communication protocol between the edge device and the upper layers. The topic structure separates telemetry, commands, acknowledgements, and configuration events. This separation is necessary because each message type has a different meaning and must be processed differently by the Data Hub and HMI.

The device-scoped topic structure allows the system to address a specific edge node. Telemetry is published from the device to the upper layers. Parameter-set messages are published through the upper-layer command path to the edge device. Acknowledgement messages are published by the edge device after it handles a parameter command. Configuration-related messages can also be extended for alarm rules, storage rules, or other supervisory settings when required.

The telemetry payload is structured as JSON and contains identity, process values, controller state, output state, timing quality, safety state, and communication state. Important fields include device identifier, run identifier, measured or simulated temperature, setpoint, error, control output, PWM value, controller coefficients, control period, sensor validity, fault state, MQTT connection state, and pending-parameter information.

The parameter-set payload contains the values that may change during runtime, such as setpoint, controller coefficients, control period, control mode, and apply semantics. Additional metadata, such as source and request time, can be stored for traceability. The acknowledgement payload contains the device identifier, acknowledgement type, success flag, applied parameter values, reason, safety state, and related runtime information.

This data model is intentionally explicit. It makes the system easier to test because every important transition can be observed in messages or storage records. It also

supports later extension because additional fields can be added without changing the main topic separation.

3.7 Auxiliary decision-support interface

The auxiliary decision-support mechanism is connected to the architecture through stored telemetry, parameter records, and HMI actions. It analyzes historical behavior and prepares parameter recommendations, but it does not control the actuator directly and does not bypass the normal command path.

The input for decision support comes from the Data Hub and HMI data sources. Historical telemetry windows, setpoint changes, actuator output, controller coefficients, acknowledgement records, and post-apply results provide the context for evaluating control behavior. These records allow the mechanism to identify patterns such as slow response, overshoot, oscillation, or poor settling behavior.

The output of decision support is a reviewable recommendation rather than an automatic control action. A recommendation may include proposed parameter values and supporting information for the operator. Before it affects the system, it must be reviewed in the HMI and applied through the same parameter-set mechanism used for manual configuration. After application, the edge device must acknowledge the command and later telemetry must be checked.

This interface keeps the auxiliary mechanism explainable and traceable. It improves the engineering value of the system by supporting parameter analysis and recommendation preparation, while the main architecture remains based on edge execution, Data Hub processing, and HMI supervision.

The architecture described in this section defines the structural basis for the implementation of the developed system. The following section focuses on the edge control layer, including temperature acquisition, local control execution, runtime parameter handling, MQTT communication, and acknowledgement processing.

4 IMPLEMENTATION OF THE EDGE CONTROL LAYER

4.1 Control target and hardware design scope

The edge control layer is the local execution part of the developed temperature-control system. Its control target is to maintain the temperature of a small chamber near the specified setpoint by measuring chamber temperature, calculating a control action, and modulating heater power through a PWM-driven output stage. In the implemented configuration, the nominal setpoint is 35 °C, while the accepted operator-configurable setpoint range is 20–60 °C. The software safety limit is 65 °C. These values define the expected laboratory prototype operating range and prevent the operator from applying a target temperature outside the intended small-chamber control task [1].

The main hardware choices connected with this control target are summarized in Table 4.1. The table is limited to the key design decisions that affect control execution, temperature feedback, power switching, supply stability, testing safety, debugging, and physical placement.

Table 4.1 – Main hardware choices for the edge control node

Functional block	**Selected hardware or solution**	**Purpose**
Controller and communication	ESP32-WROOM-32 module	GPIO, PWM, Wi-Fi/MQTT, and serial debugging for local edge control
Temperature feedback	DS18B20 sensor with 4.7 kΩ OneWire pull-up	Digital chamber-temperature feedback without analog conditioning
Heater actuation	Low-side N-channel logic-level MOSFET switch	Separates heater current from ESP32 GPIO and supports PWM power control
Power supply path	12 V input and 3.3 V buck-regulated logic rail	Supplies the load side and stable low-voltage electronics
Basic cutoff and testing	Series heater-supply cutoff in the 12 V path	Allows load-side interruption during setup and abnormal tests

Functional block	Selected hardware or solution	Purpose
Debugging and physical support	Status LED, serial interface, connectors, and separated enclosure layout	Supports assembly checks, diagnostics, and representative sensor placement

For the simulation profile used during thesis development, the expected control performance is defined as a stable approach to the setpoint without sustained oscillation, with the temperature entering a ±0.5 °C band in approximately 20 s and a ±0.2 °C band in approximately 30 s for the default 35 °C case. The 90 % rise time for the same profile is expected to be about 11 s. These values are design targets for the simulated thermal model and firmware tuning, not final measured characteristics of the physical chamber. After real assembly, the same indicators must be measured again with an external thermometer and instrumented PWM/load-current checks.

This target determines the hardware structure: the system needs a microcontroller for control execution, a temperature sensor for feedback, a power driver for the heater, a stable low-voltage supply for logic, and a mechanical enclosure that separates the thermal chamber from the electronics area. The ESP32-WROOM-32 module is selected because it provides the embedded controller, GPIO, PWM-capable peripherals, and wireless communication resources required by the edge node [3]. The DS18B20 is selected because the control range is moderate and does not require a high-temperature thermocouple [4]. The MOSFET power stage is selected because the actuator is a heater and therefore requires switched load current rather than a direct microcontroller output. The enclosure geometry is selected to make the measured chamber temperature representative of the controlled object rather than the PCB, heater surface, or local hot spot.

The hardware part of the project is therefore not an isolated addition to the software system. It follows directly from the control objective. Since the controlled variable is chamber temperature, the sensor must be placed where it represents the chamber air rather than the PCB temperature or the heater surface. Since the manipulated variable is heater power, the actuator path must be designed as a power-switching path rather than as a direct microcontroller load. Since the Data Hub and HMI require traceability, the edge hardware must also support debugging, status indication, and a communication-capable controller.

The present project contains several hardware-design artifacts: the electrical schematic exported from the circuit-design environment, the component placement screenshot, the ESP32 pin map, the Wokwi simulation connection, PCB reference files,

and a three-dimensional enclosure model. These artifacts are used to support the engineering design and to show how the simulated control node can be transferred toward a real device. At the same time, the current thesis does not claim that the final physical circuit has already been assembled and electrically tested. The real power stage and PCB must still be verified with hardware instruments after fabrication.

This boundary is important for correct engineering interpretation. The simulation verifies the logical connection between sensing, control, PWM output, telemetry, and parameter handling. The hardware design explains how this logic should be translated into an electrical and mechanical implementation. The remaining real-hardware stage must verify actual voltage levels, PWM waveform quality, MOSFET switching behavior, load current, heating response, grounding, and thermal safety under real operating conditions.

The electrical schematic of the edge temperature-control node is shown in Figure 4.1. It represents the main control, sensing, low-voltage supply, MOSFET switching, status indication, and connector paths prepared for the hardware stage.

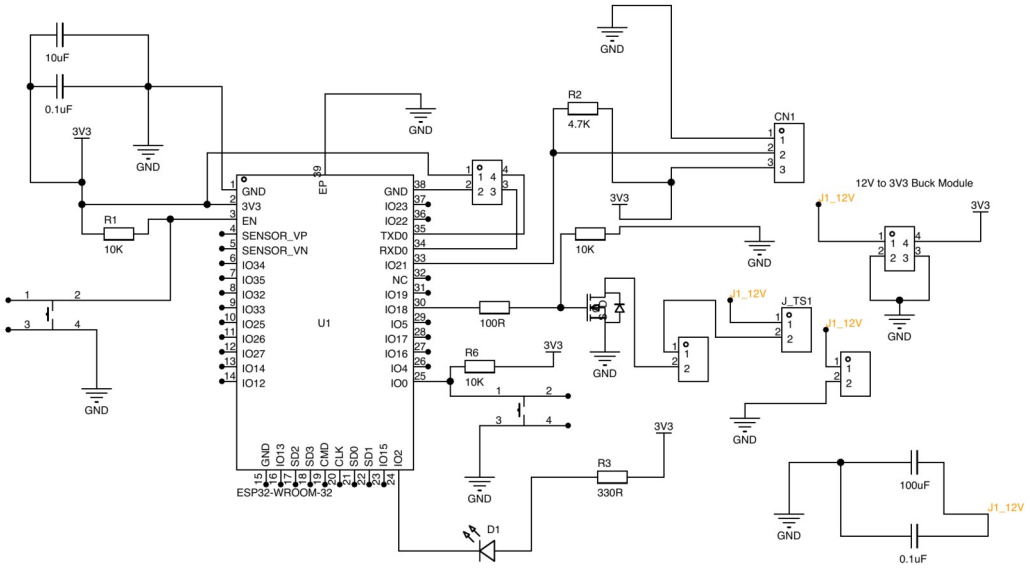

Figure 4.1 – Electrical schematic of the edge temperature-control node

4.2 Electrical schematic and pin assignment

Figure 4.1 shows the electrical schematic prepared for the edge node. The central component is the ESP32-WROOM-32 module. The DS18B20 temperature-sensor connector is connected to the ESP32 through the OneWire data line. The data line uses a 4.7 kΩ pull-up resistor to the 3.3 V rail, which is required because the OneWire bus needs a defined high level when no device is actively pulling the line low. The schematic

also includes a status LED with a current-limiting resistor, reset and boot buttons, decoupling capacitors, a 12 V to 3.3 V supply path, a heater connector, and the MOSFET switching path used for the actuator.

The selected pin assignment is consistent with the firmware implementation. GPIO21 is used for the DS18B20 data line, GPIO18 is used as the PWM output, GPIO2 is used for the heartbeat LED, and the serial TX/RX pins are used for debugging. This assignment separates the sensing path from the actuation path and leaves the PWM output available for later connection to the MOSFET gate-drive circuit. The logic analyzer is connected to the PWM signal in the simulation so that the duty-cycle behavior can be observed during controller operation.

The pull-up resistor value is chosen as a typical value for DS18B20 OneWire communication. A very large pull-up resistance would make the signal rise more slowly and could reduce communication reliability, while a very small resistance would increase unnecessary current when the bus is pulled low. For the current design, the pull-up current can be estimated by formula (4.1).

$$I_{pullup} = \frac{U_{DD}}{R_{pullup}} \tag{4.1}$$

where I_pullup – pull-up current, A;
U_DD – logic supply voltage, V;
R_pullup – pull-up resistance, Ω;

For U_DD = 3.3 V and R_pullup = 4.7 kΩ, the estimated pull-up current is approximately 0.7 mA when the line is low. This value is suitable for a low-power digital sensor interface and explains why the DS18B20 data line can be connected to the ESP32 logic domain without a high-current driver.

The Wokwi connection is still used as a simulation and firmware-verification aid [11]. It validates the logical pin mapping and the firmware interaction with the sensor, LED, PWM signal, logic analyzer, and serial monitor. However, the Wokwi model does not replace the electrical schematic, and the schematic itself does not replace physical measurement. Input protection, power regulation, grounding behavior, connector reliability, thermal protection, and load-side current must be checked during the later hardware stage.

4.3 Power path and MOSFET driver design

The actuation path is based on the idea that the ESP32 must not power the heater directly. A microcontroller pin can provide only a low-current logic signal, while a heater requires a separate power path. Therefore, the hardware design uses a low-side N-channel

MOSFET switching concept. In this arrangement, the heater is connected to the positive load supply, the MOSFET is placed between the heater and ground, and the ESP32 PWM signal controls the MOSFET gate through a gate resistor.

The choice of a MOSFET low-side switch follows from the control target. The controller needs to vary the average heating power while keeping the local control loop simple and deterministic. PWM control provides this behavior by switching the load on and off with a controlled duty ratio. The average voltage applied to an ideal resistive load over one PWM period can be represented by formula (4.2).

$$U_{avg} = D\,U_s \tag{4.2}$$

where U_avg – average load voltage over one PWM period, V;
D – PWM duty ratio;
U_s – load supply voltage, V;

For a resistive heater, the load power depends on supply voltage and load resistance. The nominal heater power can be estimated by formula (4.3).

$$P_{load} = \frac{U_s^2}{R_{load}} \tag{4.3}$$

where P_load – heater load power, W;
U_s – load supply voltage, V;
R_load – heater resistance, Ω;

These formulas explain why the heater path must be designed as a power circuit. The ESP32 only determines the duty ratio, while the actual heater current depends on the external load supply and heater resistance. For this reason, the power path must be checked separately from the firmware logic during real-hardware testing.

The MOSFET must be selected according to several engineering criteria. First, the drain-source voltage rating must exceed the load supply voltage with margin. Second, the continuous drain-current rating must exceed the expected heater current. Third, the MOSFET should have low on-state resistance at the available gate-drive voltage. Since the ESP32 gate drive is 3.3 V, the MOSFET should be logic-level and suitable for low-voltage gate operation. The conduction loss of the MOSFET can be estimated by formula (4.4).

$$P_Q = I_{load}^2 R_{DS(on)} \tag{4.4}$$

where P_Q – MOSFET conduction loss, W;
I_load – heater current, A;

R_DS(on) – MOSFET drain-source on-state resistance, Ω;

This loss estimate is needed because the MOSFET may heat during operation. If the calculated loss or measured temperature rise is too high, the design must be changed by selecting a lower-resistance MOSFET, improving thermal dissipation, reducing load current, or changing the switching arrangement. The gate resistor is included to limit fast switching edges and reduce stress or ringing on the gate path. The gate pull-down resistor is included so the MOSFET remains off during reset, startup, or a floating-control condition.

The driver concept also requires a common reference ground between the ESP32 logic and the MOSFET source. Without a common reference, the gate-source voltage would not be defined correctly. At the same time, the load-current return path must be designed carefully so that heater current does not disturb sensor measurement or controller operation. In later hardware testing, this point must be checked by measuring ground behavior and PWM switching under load.

4.4 PCB and enclosure design

The project also contains PCB reference files and a three-dimensional enclosure model. The PCB reference package includes a STEP model, a DXF board export, a BOM file, a netlist export, and the component placement screenshot shown in Figure 4.2. These files show that the board-level design has progressed beyond a simple simulation connection. They are used as mechanical and layout references for the enclosure and for future hardware integration. However, the current thesis must describe this stage accurately: the board is treated as a design reference and has not yet been verified as a fully tested physical power-control circuit.

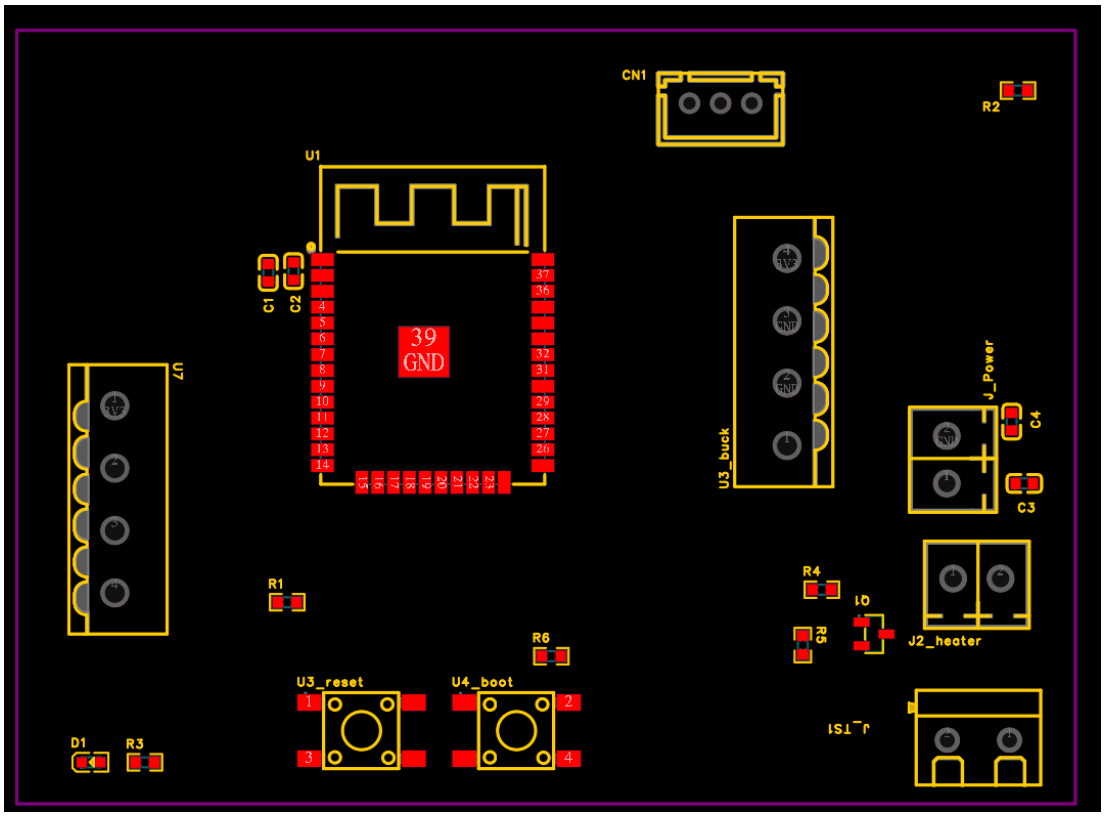

Figure 4.2 – Component placement of the edge control node

The netlist export contains the main circuit elements expected for the edge control hardware, including the ESP32 module, the DS18B20 connector, the heater connector, a power connector, a buck-regulator-related connector block, decoupling capacitors, resistors, LED indication, reset and boot buttons, and a MOSFET device. This confirms the intended hardware direction: logic control, sensor input, low-voltage supply, heater output, and service/debug support are all represented in the design package.

The PCB design must be evaluated according to the same control objective as the firmware. The heater current path should be routed with sufficient width and clearance. The MOSFET and heater connector should be located so that load wiring remains short and separated from sensitive sensor wiring. The DS18B20 connector should avoid noisy high-current paths. The regulator and decoupling elements should provide stable power to the ESP32 and sensor. These requirements come from the need to maintain reliable feedback and safe power switching during closed-loop operation.

The mechanical enclosure is designed as a small temperature-control chamber rather than as a simple PCB box. The three-dimensional layout is shown in Figure 4.3. The model separates the thermal chamber from the electronics bay. This is important because the PCB should not be placed directly in the heated volume, and the DS18B20 should measure chamber temperature rather than the board temperature or heater surface temperature.

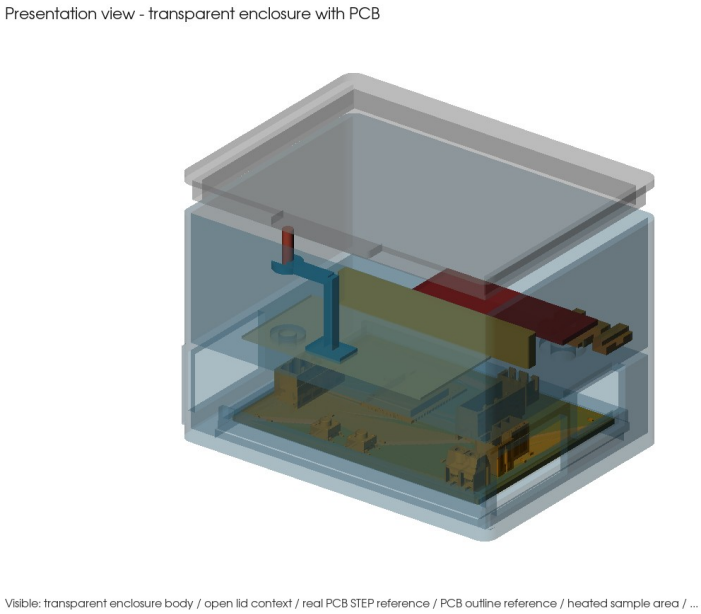

Figure 4.3 – Three-dimensional enclosure layout with PCB reference

The enclosure design includes a chamber area, a separate electronics area, PCB support features, side guide rails, a sensor probe clip, heater wire pass-through support, heater strain relief, and a thermal safety barrier between sample and heater zones. These features reflect practical control-system requirements. The sensor needs repeatable placement, the heater wiring needs mechanical support, and the electronics need to be protected from direct chamber heating. The printable enclosure parts prepared for later fabrication are shown in Figure 4.4.

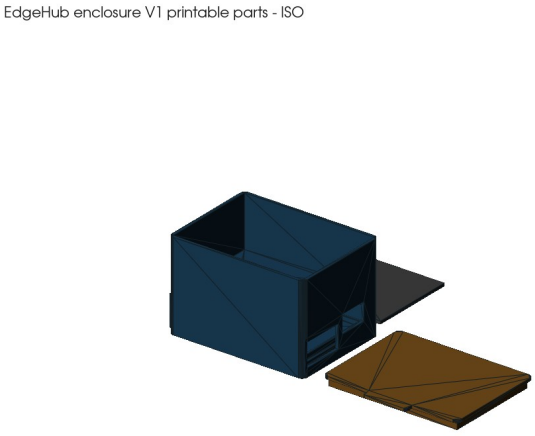

Figure 4.4 – Printable enclosure parts prepared for later fabrication

The enclosure is therefore part of the control design, not only a cosmetic shell. If the sensor is placed too close to the heater, the measured temperature may represent local heater temperature rather than chamber temperature. If the electronics are exposed to the

heated volume, the ESP32, connectors, or power components may operate under unsuitable thermal conditions. If the heater wires are not constrained, mechanical movement may affect reliability. The enclosure geometry is designed to reduce these risks at the prototype stage.

The next hardware stage must include physical fabrication and measurement. After the PCB and enclosure are assembled, the circuit should be tested with a multimeter for supply rails and continuity, an oscilloscope or logic analyzer for PWM waveform quality, and a controlled load test for MOSFET switching and heater current. The DS18B20 reading should be compared with an external reference thermometer, and the chamber response should be recorded during heating and cooling. Only after these tests can the hardware be claimed as electrically and thermally validated.

4.5 Temperature measurement, feedback, and local control algorithm

The firmware implementation connects the hardware design to the closed-loop control behavior. At each control tick, the edge application reads the DS18B20 temperature value, checks sensor validity, selects the active feedback value, calculates the controller output, applies the PWM duty cycle, updates the simulation plant model when the simulator profile is active, and publishes telemetry. This sequence makes the ESP32 node the local execution core of the system.

The developed implementation distinguishes between physical sensor reading and simulated controlled temperature. In Wokwi, the DS18B20 value is available as a sensor reference, but the virtual heater output does not physically heat the DS18B20 model. Therefore, the simulator profile uses a virtual thermal model as the controlled process variable while still reporting the DS18B20 reading. This decision is important because it avoids claiming a physical heating effect that the simulator cannot provide.

The virtual thermal model represents heating as a function of normalized PWM duty and cooling as a function of the difference between simulated temperature and ambient temperature. It is intentionally simple, but it is sufficient for repeatable control experiments. The model lets the project demonstrate rise time, convergence, output saturation, parameter influence, and post-apply behavior without requiring a physical heater during early development.

The controller is PI-oriented with a PID-compatible runtime parameter structure. The runtime configuration contains proportional, integral, and derivative coefficients, but the default derivative coefficient is zero. This means the tuned default behavior is PI-like while the message contract remains compatible with PID-style parameter storage and future experiments. The control error is calculated by formula (4.5).

$$e(t) = T_{set} - T(t) \tag{4.5}$$

where e(t) – temperature-control error, °C;

T_set – target temperature, °C;

T(t) – measured or simulated controlled temperature, °C;

The general continuous PID control law used as the reference form is given by formula (4.6).

$$u(t) = K_p\ e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d\ \frac{d\ e(t)}{dt} \tag{4.6}$$

where u(t) – controller output before limiting;

K_p – proportional gain;

K_i – integral gain;

K_d – derivative gain;

The initial controller coefficients were not selected as arbitrary software constants. The engineering basis is the classical Ziegler-Nichols closed-loop method, also known as the ultimate-gain method, which is widely used as an initial PID tuning procedure [13]. In this procedure the integral and derivative actions are first disabled, the proportional gain is increased until the controlled process approaches sustained oscillation, and the observed ultimate gain and oscillation period are used to calculate the initial PID coefficients. For the parallel PID form in formula (4.6), the corresponding initial relations are given by formula (4.7).

$$K_p = 0.6\,K_u;\ \ K_i = \frac{1.2\,K_u}{T_u};\ \ K_d = 0.075\,K_u T_u \tag{4.7}$$

where K_u – ultimate proportional gain obtained from the onset of sustained oscillation;

T_u – ultimate oscillation period, s;

In the present project this method was used as the engineering reference for the first tuning stage. Since the physical chamber and power stage had not yet been assembled and instrumented, the final ultimate gain and ultimate period must be confirmed later on the real device by a controlled closed-loop test. At the simulator stage, the Ziegler-Nichols-based estimate was intentionally converted into a conservative PI-oriented setting because the thermal object is slow, the actuator is PWM-limited, and the DS18B20 measurement is discrete.

In the implemented default configuration, K_p = 120, K_i = 12, and K_d = 0. This value set should therefore be interpreted as an initial engineering tuning refined by

simulation rather than as a final industrial tuning of the physical object. The derivative part is available in the runtime structure but is not active in the tuned default behavior. The proportional term reacts to current error, while the integral term helps reduce steady-state error. The selected integral gain was reduced from the more aggressive initial PI test value after simulation showed that the lower value preserved final accuracy while avoiding unnecessary overshoot. The calculated output is limited to the PWM range from 0 to 255, and the normalized duty ratio sent to the actuator path is calculated by formula (4.8).

$$D = \frac{u}{255} \tag{4.8}$$

where D – normalized PWM duty ratio;

u – limited controller output;

Several implementation details improve embedded-control behavior. The controller uses the measured elapsed time between control ticks, clamps the output to the valid PWM range, limits the integral state, and avoids increasing the integral term when the output is already saturated in the same direction as the error. This is a practical anti-windup measure. The output saturation state is included in telemetry so that the Data Hub and HMI can explain slow response or limited control authority.

The key part of the implemented anti-windup logic is shown in Figure 4.5. The fragment demonstrates that the integral candidate is calculated first, but it is committed only when the controller output is not saturated in the same direction as the current error. The same fragment also shows how the raw controller output is limited to the PWM range and how the saturation state is made explicit for later telemetry analysis.

Source fragment: simulator/wokwi/src/controller/pi_controller.cpp, lines 31–60

```
  const float candidate_integral = constrain(
      integral_error_ + out.error_c * safe_dt_s,
      cfg_.integral_min,
      cfg_.integral_max);

  const float candidate_output = out.error_c * runtime.kp + candidate_integral * active_ki + out.d_term;
  const bool saturating_high = candidate_output > cfg_.max_duty && out.error_c > 0.0f;
  const bool saturating_low = candidate_output < 0.0f && out.error_c < 0.0f;

  if (!(saturating_high || saturating_low)) {
    integral_error_ = candidate_integral;
  }

  out.integral_error = integral_error_;
  previous_error_ = out.error_c;
  previous_error_initialized_ = true;

  const float raw_output = out.error_c * runtime.kp + out.integral_error * active_ki + out.d_term;
  if (raw_output < 0.0f) {
    out.saturation_state = edge::domain::SaturationState::kLow;
  } else if (raw_output > cfg_.max_duty) {
    out.saturation_state = edge::domain::SaturationState::kHigh;
  } else {
    out.saturation_state = edge::domain::SaturationState::kNone;
  }

  out.control_output = constrain(raw_output, 0.0f, static_cast<float>(cfg_.max_duty));
  out.pwm_duty = static_cast<uint8_t>(out.control_output);
  out.pwm_norm = static_cast<float>(out.pwm_duty) / static_cast<float>(cfg_.max_duty);
  return out;
```

Figure 4.5 – PID output limiting and anti-windup implementation fragment

When a new runtime configuration is applied, the controller integral state can be reset. This prevents hidden accumulated error from continuing to influence the process after the operator changes the setpoint or gains. This behavior supports post-apply verification because the observed response after a parameter update is more directly connected to the new active configuration.

4.6 Runtime configuration, MQTT handling, and safety validation

The edge node maintains an active runtime configuration that includes the target temperature, controller gains, control period, and control mode. During operation, this configuration can be changed through the MQTT parameter command path. A parameter message is parsed, validated, applied immediately or staged, and followed by an acknowledgement. This behavior is essential because a command published by an upper layer is not the same as a command accepted by the device.

The parser supports partial parameter updates. For example, a message may update only the setpoint or only one controller coefficient while leaving the remaining parameters unchanged. The validator checks whether the target temperature, gains, control period, and control mode are within allowed ranges. If parsing or validation fails, the device does not apply the command and publishes a failure acknowledgement with a reason. If validation succeeds, the device either applies the update immediately or stores it as pending.

Telemetry is published after control ticks as structured JSON. It includes identity, uptime, setpoint, simulated and sensor temperature, control error, integral error, derivative information, PWM duty, normalized PWM output, saturation state, control period, timing error, controller parameters, Wi-Fi state, MQTT state, fault state, safety-output state, and pending-parameter state. These fields allow the Data Hub and HMI to reconstruct not only what temperature was measured, but also why the controller behaved in a particular way.

The acknowledgement payload contains the acknowledgement type, success flag, applied-immediately flag, pending-parameter state, active runtime parameters, reason, uptime, sensor validity, fault state, and software safety limit. This gives the HMI a reliable way to show whether a parameter command was processed by the edge node. It also gives the Data Hub a persistent record that can be compared with later telemetry during post-apply verification.

Safety behavior is implemented locally because the edge node is the final execution point for the actuator. If the sensor value is invalid, the firmware latches a fault. If the measured sensor temperature exceeds the configured software maximum safe temperature, the firmware also latches a fault. In a fault state, the controller integral is reset and the actuator output is forced to zero. The telemetry then reports the fault reason and the safety-output state.

The firmware fragment shown in Figure 4.6 connects measurement, safety checking, local control, actuator output, simulator update, telemetry construction, and MQTT publication. This fragment is important because it demonstrates that the edge layer is not only a mathematical controller, but a complete local execution loop with safety forcing and traceable output state.

Source fragment: simulator/wokwi/src/app/edge_app.cpp, lines 220–267

```
  const float sensor_temp = sensor_.read_celsius();
  state_.sensor_valid = sensor_.is_valid(sensor_temp);
  state_.sensor_temp_c = sensor_temp;

  if (!state_.sensor_valid) {
    latch_fault("sensor_invalid");
  } else if (state_.sensor_temp_c > cfg_.control.software_max_safe_temp_c) {
    latch_fault("software_overtemp");
  }
  clear_fault_if_possible();

  const edge::domain::TemperatureSample feedback =
      feedback_selector_.select(state_.sensor_temp_c, state_.sensor_valid,
                                state_.simulated_temp_c);

  edge::domain::ControllerInput input;
  input.target_temp_c = runtime_store_.current().target_temp_c;
  input.measured_temp_c = feedback.temperature_c;
  input.dt_s = static_cast<float>(elapsed_ms) / 1000.0f;
  if (input.dt_s <= 0.0f) {
    input.dt_s = runtime_store_.current().control_period_ms / 1000.0f;
  }

  bool safety_output_forced_off = state_.fault_latched;
  edge::domain::ControllerOutput control{};
  if (safety_output_forced_off) {
    control.error_c = input.target_temp_c - input.measured_temp_c;
    control.control_output = 0.0f;
    control.pwm_duty = 0;
    control.pwm_norm = 0.0f;
    control.saturation_state = edge::domain::SaturationState::kLow;
    actuator_.write_duty(0);
  } else {
    control = controller_.update(input, runtime_store_.current());
    actuator_.write_duty(control.pwm_duty);
  }

#if EDGE_BUILD_SIMULATOR
  state_.simulated_temp_c =
      plant_model_.step(state_.simulated_temp_c, control.pwm_norm);
#endif

  const edge::domain::TelemetrySnapshot snapshot =
      build_snapshot(now_ms, feedback, control, safety_output_forced_off);

  const String telemetry_json = telemetry_builder_.to_json(snapshot);
  Serial.println(telemetry_json);
  mqtt_.publish_telemetry(snapshot);
```

Figure 4.6 – Edge control tick with safety forcing and telemetry publishing

The local safety behavior does not replace real electrical protection. It is a software safety layer that must be supported by proper hardware design and measurement. During later hardware testing, the PWM output should be checked with an oscilloscope, the MOSFET gate voltage should be measured under switching conditions, the heater current should be measured under load, the regulator output should be checked during switching, and the chamber temperature should be compared with an external thermometer. These tests are necessary before the PCB and enclosure can be considered validated as real hardware.

The planned hardware-verification sequence should therefore start from low-risk checks and move gradually toward closed-loop operation. First, the assembled PCB should be inspected visually to confirm component orientation, solder quality, connector placement, and absence of visible shorts. Second, continuity and resistance checks should

be performed with the power disconnected, especially on the 12 V rail, the 3.3 V rail, the heater output path, and the ground network. Third, the low-voltage supply should be powered without the heater load, and the 3.3 V rail should be measured under static and switching conditions.

After the supply checks, the control-output path should be verified separately from the thermal chamber. The ESP32 PWM output should be observed with an oscilloscope or logic analyzer at the microcontroller pin and then at the MOSFET gate. The measured duty ratio should correspond to the controller command, and the waveform should not show unexpected pulses during reset or startup. A resistive test load can then be connected instead of the final heater so that MOSFET switching, load current, voltage drop, and component heating can be checked under controlled conditions.

The final hardware stage should connect the electrical behavior to the controlled object. The DS18B20 reading should be compared with an external thermometer at several chamber temperatures, and the sensor position should be adjusted if it measures local heater influence rather than representative chamber air. The heater should then be tested with conservative duty limits before the full controller is enabled. During this stage, recorded telemetry, acknowledgements, and HMI observations should be compared with instrument measurements. This comparison is necessary because a software log alone cannot prove that the physical circuit behaves safely.

These verification steps define the boundary between the completed design work and the remaining physical validation work. In the present thesis, the edge layer is implemented and prepared for hardware transfer, while the final claim of electrical and thermal validation is intentionally reserved for the later assembled prototype.

The implemented edge layer therefore connects the hardware design, control algorithm, runtime configuration, MQTT communication, and safety behavior into one local execution subsystem. It provides the foundation for the Data Hub and HMI implementation described in the following chapter.

5 IMPLEMENTATION OF THE DATA HUB AND HMI LAYERS

5.1 MQTT ingestion pipeline

The Data Hub layer is the central processing part of the developed temperature-control and monitoring system. Its role is to receive messages from the edge control layer, transform them into stable engineering records, maintain device state, and provide stored data for the HMI layer and auxiliary decision-support mechanism. In the implemented system, the Data Hub is developed as a Java Spring Boot application [5]. This choice separates the continuous message-ingestion workload from the HMI user interface and makes the Data Hub responsible for the technical path between MQTT communication and persistent storage.

The ingestion pipeline subscribes to the device-scoped MQTT topics used by the edge node. These topics include telemetry messages, parameter-set messages, and parameter-acknowledgement messages. The implemented topic filters are based on the structure edge/temperature/+/telemetry, edge/temperature/+/params/set, and edge/temperature/+/params/ack. This structure allows the same Data Hub application to process messages from one simulated node or from several future physical nodes without changing the main processing logic.

The pipeline is implemented as a bounded asynchronous processing flow. MQTT messages are accepted by the source component, converted into internal envelopes, and passed to a Reactor-based pipeline [6]. The pipeline uses buffering, configurable prefetching, and device-based lane partitioning. This is important because telemetry is periodic and may arrive continuously, while parameter commands and acknowledgements are event-like and must not be lost or hidden by a large telemetry stream. The bounded queue design also prevents uncontrolled memory growth when message production temporarily exceeds processing capacity.

The central part of this implementation is shown in Figure 5.1. The fragment demonstrates the bounded buffer, configured overflow behavior, parallel scheduler, and fixed device-lane partitioning used to process MQTT messages without relying on an uncontrolled queue.

Source fragment: data-hub/src/main/java/com/edgehub/datahub/pipeline/MqttConsumePipeline.java, lines 169–187

```
169      return source.messages()
170          .onBackpressureBuffer(
171              pipelineBufferSize,
172              this::logPipelineDrop,
173              overflowStrategy)
174          .publishOn(Schedulers.parallel(), prefetch)
175          // Fixed lane partitioning avoids unbounded groupBy(deviceId) starvation:
176          // with many active devices, finite flatMap concurrency can permanently starve later groups.
177          .groupBy(envelope -> laneForDevice(envelope.deviceId(), laneParallelism))
178          .flatMap(group -> group.concatMap(this::processEnvelope), laneParallelism)
179          .doOnSubscribe(ignored -> log.info(
180              "mqtt consume pipeline stream started laneParallelism={} prefetch={} pipelineBufferSize={}
     overflowStrategy={}",
181              laneParallelism,
182              prefetch,
183              pipelineBufferSize,
184              overflowStrategy))
185          .doOnError(error -> log.error("mqtt consume pipeline fatal error", error))
186          .retry()
187          .subscribe();
```

Figure 5.1 – Data Hub bounded MQTT ingestion implementation fragment

The message-processing sequence includes parsing, application of persistence policy, writing to the selected storage backend, and acknowledgement of the MQTT message after processing. When parsing or persistence fails, the error is recorded in metrics and logs. The implemented behavior is intentionally observable: accepted messages, dropped messages, parse failures, persistence failures, telemetry skips, stored parameter commands, stored acknowledgements, device-status events, and summary records are counted by the Data Hub metrics. This makes the ingestion layer suitable for engineering debugging rather than only for demonstration.

The Data Hub does not directly generate control actions and does not replace the HMI or the operator. Its responsibility is to maintain a reliable data path and to preserve the facts needed for later supervision. The relationship between the main Data Hub processing functions and their engineering purpose is summarized in Table 5.1.

Table 5.1 – Main Data Hub processing responsibilities

Function	**Implemented mechanism**	**Engineering purpose**
MQTT ingestion	Device-scoped subscriptions for telemetry, parameter commands, and acknowledgements	Continuous reception of edge-layer messages
Message parsing	Topic classification and JSON payload mapping into stable models	Separation of raw MQTT exchange from internal data processing

Function	Implemented mechanism	Engineering purpose
Bounded processing	Reactor pipeline with configurable buffers, prefetch, and lane partitioning	Stable operation under burst traffic and multiple devices
Persistence handoff	Writer abstraction for log, file archive, or TDengine REST storage	Reproducible data storage and later analysis
Device state tracking	Online/offline state updates based on received messages and timeouts	HMI visibility of device availability
Operational metrics	Counters for received, parsed, skipped, persisted, and failed messages	Runtime observability and troubleshooting

5.2 Message parsing and topic model

The implemented message parser uses both the MQTT topic and the JSON payload. The topic identifies the device and the message class, while the payload contains the data fields. This separation is important because a topic alone cannot describe the control state, and a payload alone is not sufficient to route the message correctly. The Data Hub therefore treats every received MQTT message as an envelope that contains the raw topic, the received timestamp, and the payload body.

The parser maps telemetry messages to the telemetry payload model. The telemetry payload contains the device identifier, uptime, target temperature, simulated temperature, sensor temperature, control error, integral error, controller output, PWM duty, normalized PWM value, control period, saturation state, controller mode, controller version, PID coefficients, communication state, sensor validity, and safety state. These fields correspond to the edge-layer telemetry contract and allow the upper layers to reconstruct not only the measured temperature, but also the reason for the controller output.

Parameter-set messages are parsed separately from telemetry. They represent an upper-layer intent to change runtime configuration. The message may contain a target temperature, controller coefficients, control mode, control period, and an apply_immediately flag. The Data Hub stores this intent as a separate fact because the publication of a command does not prove that the device accepted it. This design avoids a common weakness in simple monitoring prototypes, where an operator action is treated as successful before the controlled device has confirmed it.

Parameter-acknowledgement messages are also parsed as a separate message class. An acknowledgement contains the result of the edge-side parsing and validation step, including success status, acknowledgement type, active parameter values, reason, sensor validity, fault state, and safety limit. This record closes the command side of the engineering loop. It allows the HMI and the stored history to distinguish between a requested change, a rejected change, a staged change, and an applied change.

The parser is tolerant of unknown JSON fields. This is useful in the project because telemetry evolves during development: additional diagnostic fields can be added by the edge node without immediately breaking Data Hub ingestion. At the same time, the fields used for engineering logic remain explicit. The result is a stable but extensible interface between the edge control layer and the Data Hub layer.

5.3 Telemetry filtering, aggregation, and storage

Persistent storage is required because a temperature-control system cannot be evaluated only from the current screen value. The relevant engineering questions depend on history: how quickly the temperature approached the setpoint, whether overshoot occurred, whether the output saturated, which parameter version was active, and whether the behavior changed after a configuration command. For this reason, the Data Hub stores telemetry, parameter commands, acknowledgements, summaries, alarm facts, and device-status records.

The implemented storage design supports several modes. During development, messages can be logged or archived to local files for inspection. For time-series operation, the Data Hub can write normalized records to TDengine through the REST SQL interface [7]. TDengine is used for high-frequency telemetry and event history, while the HMI backend uses PostgreSQL for relational control-plane data such as users, devices, parameters, roles, rules, recommendations, and other application objects [12]. This separation keeps time-series facts and business objects in different storage areas.

Telemetry writing can be performed directly or through a batch path. In the TDengine REST writer, telemetry batching groups rows by device-related table lanes and flushes them by row count or time delay. This reduces the overhead of frequent single-row writes while keeping the storage delay bounded. The writer also creates separate records for telemetry summaries, parameter-set events, parameter acknowledgements, alarm events, and device status.

Filtering and aggregation are used to reduce unnecessary storage noise when the process is steady. The telemetry write filter can skip repeated stable samples according to configurable deadbands, while the summary aggregator can produce compact steady-state windows. This does not remove important engineering information because command

events and acknowledgement events invalidate the filter state and force relevant data to be preserved around configuration changes. In other words, storage reduction is allowed only when it does not hide meaningful control behavior.

The summary data supports later analysis. A summary window stores values such as average temperature, average error, maximum absolute error, PWM range, controller parameters, control period, and flush reason. These fields are useful for HMI history views and for auxiliary decision-support preparation. The storage design therefore supports both immediate monitoring and later evaluation of control quality.

5.4 Alarm and rule processing

The Data Hub includes rule-related processing for abnormal conditions and device state. The purpose of this mechanism is not to make autonomous control decisions, but to make abnormal situations visible and traceable. Examples of relevant conditions include sensor invalidity, fault-latched state, safety-output-forced-off state, high temperature, missing telemetry, or an offline device.

Alarm and storage rules are treated as supervisory configuration. The Data Hub can receive configuration-update notifications and refresh rule information from the rule store. This architecture allows alarm or storage behavior to be adjusted without changing the edge firmware. However, the current thesis should not overstate this as a complete industrial alarm-management product. In this project, the rule mechanism is implemented as an engineering support component for monitoring, visibility, and later extension.

Device-status tracking is based on message activity and timeout rules. When messages are received from a device, the Data Hub can update the device as online. If no messages arrive within the configured timeout, an offline status can be generated. This is important for the HMI because a stale last temperature value should not be interpreted as a live controlled state. The operator must be able to distinguish between a stable process and a disconnected device.

The alarm and status records are stored together with telemetry and command history. This provides context for later review. For example, if a parameter command is sent but no acknowledgement is received, the stored status events can show whether the device became unavailable. If a fault is reported after a command, the alarm history and telemetry history can be compared to understand the sequence of events. This traceability is part of the closed engineering workflow implemented by the project.

5.5 HMI backend and frontend

The HMI layer provides the operator-facing part of the system. It is implemented with a FastAPI backend and a React frontend [8], [9]. The backend exposes REST endpoints for devices, telemetry history, parameters, alarms, storage rules, users, and operational views. It also connects to TDengine for telemetry and event history and to PostgreSQL for relational control-plane data. The frontend presents this data as monitoring and configuration screens.

The backend is not a passive relay. It enforces authentication and role-based access, loads accessible devices for the current user, reads current state and history, and prepares responses for the frontend. In TDengine mode, device detail and history pages read time-series facts from TDengine. In PostgreSQL-backed areas, the backend manages users, devices, parameters, alarm rules, recommendations, and other application state. This makes the backend the controlled interface between stored data and operator actions.

The React frontend includes device overview, device detail, history, alarms, storage-rules, user-management, and operations pages. The device detail page is especially important for the thesis because it connects monitoring and control. It displays current temperature, target temperature, PWM output, alarms, historical chart data, control evaluation, runtime parameters, and command feedback. The operator can observe whether the device is close to the setpoint, whether the actuator is saturated, whether the control behavior is stable, and whether the device has recently acknowledged a parameter update.

The operator-facing monitoring and configuration screen is shown in Figure 5.2. It combines process history, target band visualization, current parameters, editable PID fields, and acknowledgement-related feedback in one view, which supports the closed-loop interpretation of operator actions.

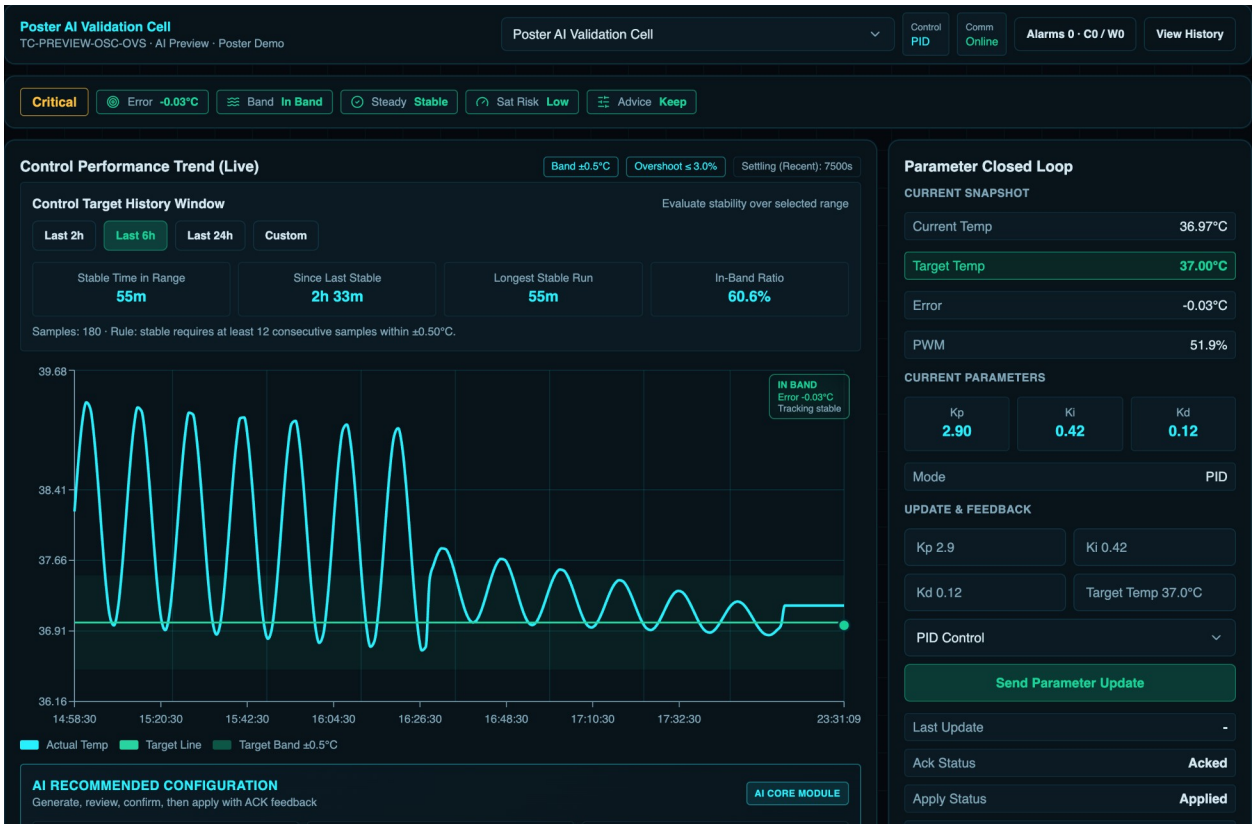

Figure 5.2 – HMI device detail screen for monitoring and parameter configuration

History views are used to examine behavior over a selected time range. They support the engineering interpretation of the control process: the operator can compare temperature, setpoint, error, PWM duty, and alarm events before and after a configuration change. This is necessary because a single current value cannot prove that control quality is acceptable. A system may have the correct current temperature but still show overshoot, long settling time, or actuator saturation in the recent history.

The HMI also includes an operations view for runtime observability. This view can show Data Hub-related metrics, backend status, model/runtime status, and log-derived indicators. In the scope of the diploma project, this is useful because the system is not a single embedded program; it is a layered platform where MQTT ingestion, storage, backend APIs, and frontend screens must work together. Operational visibility helps verify whether a failure is located in the edge node, MQTT path, Data Hub, storage layer, backend, or browser interface.

The operations console shown in Figure 5.3 is used for Data Hub and ingestion monitoring. This screen is not an operator control screen; it is an engineering view that helps inspect MQTT ingress rate, processing rate, queue depth, and runtime status during development and validation.

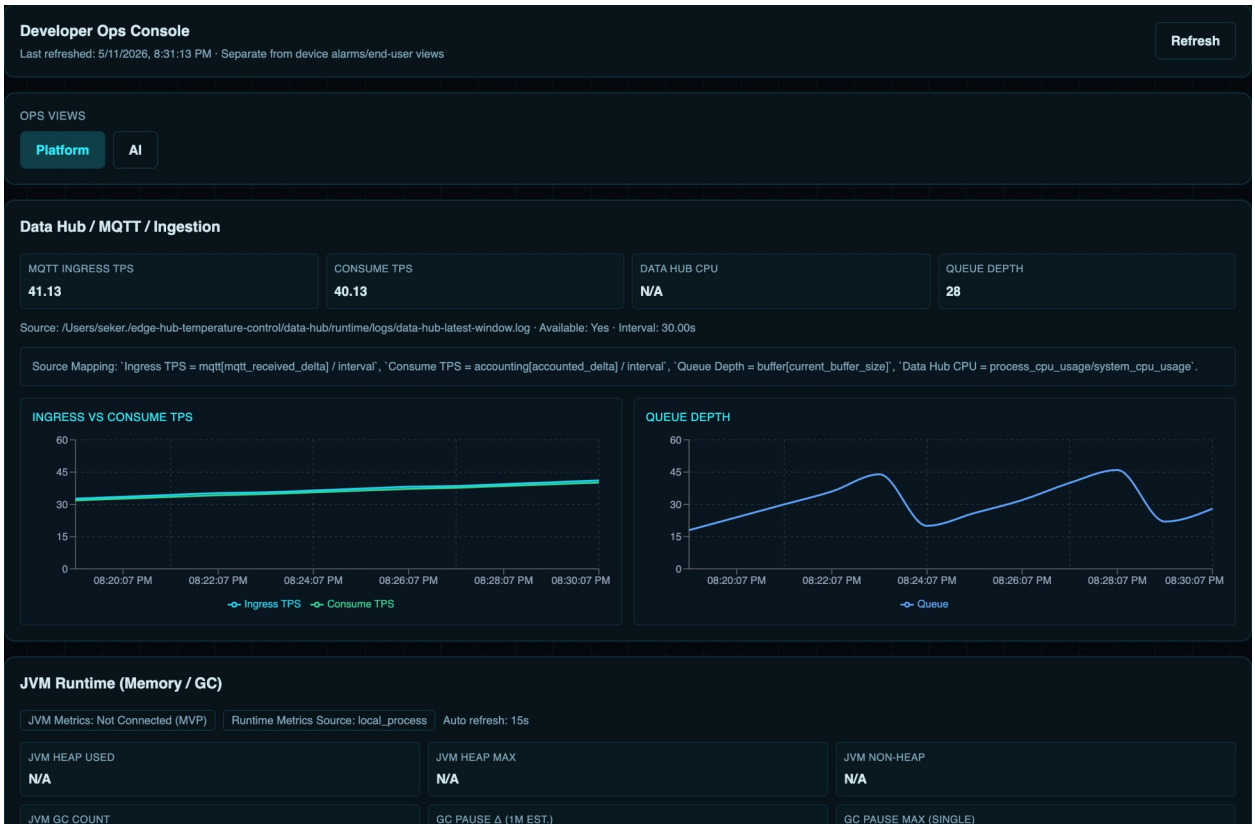

Figure 5.3 – HMI operations console for Data Hub and ingestion monitoring

5.6 Device monitoring and parameter configuration

The parameter-configuration path is the most important HMI function for the closed-loop workflow. The operator edits the target temperature or controller parameters in the HMI. The HMI frontend sends the request to the backend API. The backend then publishes the corresponding params/set message through the configured MQTT command path. This distinction is important: the frontend is the operator interface, but it is not treated as a direct MQTT client in the implemented architecture.

The backend MQTT publisher constructs a JSON payload with the selected runtime parameters, control mode, control period, apply_immediately flag, source information, and request timestamp. It publishes this payload to the device-specific parameter-set topic. The edge node then parses and validates the message. If the payload is valid, the device applies or stages the new parameters and publishes a parameter acknowledgement. If the payload is invalid, the device publishes a failure acknowledgement or reports the reason for rejection.

The command-publishing fragment is shown in Figure 5.4. It demonstrates that the HMI backend constructs a structured parameter payload, removes fields that were not changed by the operator, marks the source as HMI, and publishes the result through the device-specific MQTT command path.

Source fragment: hmi/backend/app/services/mqtt_publisher.py, lines 41–55

```
        topic = settings.mqtt_params_set_topic_template.format(device_id=device_id)
        payload_obj = {
            "target_temp_c": target_temp_c,
            "kp": kp,
            "ki": ki,
            "kd": kd,
            "control_mode": control_mode,
            "control_period_ms": control_period_ms,
            "apply_immediately": apply_immediately,
            "source": "hmi",
            "requested_at": datetime.now(tz=timezone.utc).isoformat(),
        }
        payload_obj = {k: v for k, v in payload_obj.items() if v is not None}
        payload = json.dumps(payload_obj, ensure_ascii=True, separators=(",", ":"))
        return self.publish_raw(topic=topic, payload=payload)
```

Figure 5.4 – HMI backend parameter command publishing fragment

The HMI backend supports acknowledgement-aware confirmation. After publishing a parameter command, it can query the latest params_ack records from TDengine and compare the acknowledgement timestamp and parameter values with the requested values. A strict path waits for an acknowledgement after the command time. A bounded relaxed path can also accept a recent matching acknowledgement when there is small clock skew between producer and consumer timestamps. This design is more reliable than simply reporting success after the MQTT publish operation returns.

The operator feedback is therefore based on several separate facts: the command was requested, the MQTT publish path accepted the message, the device acknowledged the command, and later telemetry shows the applied behavior. These facts are intentionally not collapsed into one status. Separating them makes the system more explainable and supports post-apply verification.

The HMI layer also keeps the operator in the loop for safety and clarity. Parameter values can be reviewed before application, command feedback can be shown after application, and history can be checked after the new parameters are active. The auxiliary decision-support mechanism, discussed later in Chapter 6, uses the same controlled path when recommendations are applied. It does not bypass the HMI and does not replace the operator.

The monitoring screen and the parameter screen therefore have to be interpreted together. The monitoring screen shows the current and historical behavior of the controlled process, while the parameter screen changes the runtime configuration that can influence this behavior. If these functions were separated without acknowledgement and history, the operator could change a parameter but would not have enough evidence to confirm whether the device actually applied the new value. In the developed implementation, the command path, acknowledgement record, and subsequent telemetry are kept as connected parts of the same engineering action.

This design also supports error diagnosis. If the HMI sends a parameter update but the device does not react, the operator can distinguish several possible causes. The MQTT publish operation may fail, the edge node may reject the payload, the acknowledgement may be missing, or the command may be applied but the thermal process may respond slowly because of inertia or actuator saturation. Each of these cases has a different engineering meaning. The implemented Data Hub and HMI layers preserve enough information to separate communication failure, validation failure, device unavailability, and normal slow thermal response.

Role separation is also important in the HMI implementation. Monitoring can be made available to users who only need to observe the process, while parameter modification is restricted to users with the appropriate operator or administrator role. This prevents the HMI from becoming an uncontrolled command interface. For the diploma prototype, this access model demonstrates the principle of controlled supervisory operation, even though a future production deployment would require a more complete security review, audit policy, and network-hardening procedure.

The backend API also provides a stable boundary for future extension. New frontend screens can use the same device, history, alarm, and parameter endpoints without changing the edge firmware. Additional storage rules, alarm rules, or decision-support views can be added above the same Data Hub records. This is consistent with the general project architecture: time-critical control remains in the edge control layer, message processing and storage remain in the Data Hub layer, and operator interaction remains in the HMI layer.

From the engineering point of view, the main result of the HMI implementation is that parameter configuration becomes a traceable operation instead of a simple screen action. The operator can see the current state, decide whether a change is needed, submit the configuration request through the backend, receive feedback about the command path, and then compare subsequent measurements with the expected behavior. This order is important for a temperature-control system because the physical process does not react instantly. Thermal inertia, actuator saturation, and measurement noise may delay the visible response even when the command has been accepted correctly.

The implemented upper layers also reduce ambiguity during testing. If the measured temperature does not approach the setpoint after a parameter update, the stored records make it possible to inspect the command event, acknowledgement event, active parameter values, controller output, and device status in one context. As a result, the developer can separate an incorrect controller setting from a communication problem or from an unavailable device. This supports the later validation stage, where the complete

path has to be checked from generated telemetry to HMI observation after the applied change.

The implementation described in this chapter completes the upper part of the layered system. The Data Hub receives and stores structured telemetry, parameter intents, acknowledgements, alarm facts, and device status. The HMI presents this information to the operator and provides a controlled parameter-configuration path. Together with the edge control layer described in Chapter 4, these layers form the implemented closed-loop workflow from measurement to operator supervision and post-apply observation.

The screenshots and source-code fragments included in this chapter also define the practical evidence boundary of the implementation. The HMI figures demonstrate that the operator-facing layer contains both process supervision and engineering observability, while the backend fragment shows that parameter commands are issued through a structured service path instead of an uncontrolled browser-side MQTT connection. The Data Hub fragment shows the bounded ingestion mechanism that protects processing from becoming an unlimited message queue. These implementation details are important for the following validation chapter because they provide concrete points at which the system can be checked: message reception, storage, HMI display, command publication, acknowledgement handling, and observation after a parameter change.

This separation of responsibilities is also useful for assessing the quality of the implementation. The edge layer can be evaluated by checking whether it produces correct telemetry and acknowledgement messages. The Data Hub can be evaluated by checking whether it preserves these messages as normalized records. The HMI can be evaluated by checking whether the operator can see the current state, review history, apply parameters, and inspect command results. The next chapter uses this separation to validate the complete loop without confusing local control execution with supervisory analysis.

6 AUXILIARY DECISION-SUPPORT AND SYSTEM VALIDATION

6.1 Purpose of decision support

The purpose of the auxiliary decision-support mechanism is to help the operator interpret stored control behavior and prepare reviewable parameter recommendations. It is not the main controller of the developed system. Local measurement, control calculation, and actuator output remain in the edge control layer, while the Data Hub and HMI provide the supervisory and data-processing environment around this local loop.

In the developed system, decision support is connected to historical telemetry, parameter records, acknowledgement records, and HMI actions. Its input is therefore not an isolated temperature value, but a window of engineering context that describes how the controlled process behaved before and after a parameter change. The mechanism analyzes this context and produces a recommendation that can be reviewed by the operator. If the recommendation is accepted, it is applied through the same HMI and MQTT command path as a manual parameter update.

This boundary is important for safety and explainability. The mechanism can identify possible slow response, steady-state error, overshoot, oscillation, or saturation-limited behavior, but it does not directly write to the actuator and does not bypass acknowledgement. The operator remains responsible for applying the change and checking subsequent measurements. The auxiliary mechanism therefore supports the layered closed-loop workflow instead of replacing it.

6.2 Data preparation and feature extraction

The decision-support mechanism requires prepared data rather than raw messages alone. Telemetry records are first collected by the edge node, delivered through MQTT, parsed by the Data Hub, and stored with timestamps, setpoints, measured or simulated temperature values, controller output, PWM duty, saturation state, controller parameters, and device status. This stored history makes it possible to calculate behavior indicators over a defined observation window.

The main extracted indicators are selected to match control-engineering meaning. Mean error and mean absolute error describe the deviation from the setpoint. Error standard deviation and temperature swing describe oscillation or unstable behavior. In-band ratio describes how much of the observation window remains inside the accepted target band. Overshoot describes how far the process exceeds the target. Settling time describes how long the response takes to remain inside the target band. Saturation ratio

describes how often the actuator output reaches the configured limit. The indicators used in the developed mechanism are summarized in Table 6.1.

The feature-extraction implementation is shown in Figure 6.1. It calculates the indicators from a telemetry window and returns them as a structured feature set. This design keeps the recommendation mechanism explainable: each later decision can be traced to measurable control-behavior features rather than to an opaque screen action.

Source fragment: hmi/backend/app/services/ai/feature_extractor.py, lines 37–78

```python
def extract_features(payload: RecommendationGenerateInput) -> FeatureSet:
    points = payload.history_window.points
    if not points:
        return FeatureSet(
            mean_error=0.0,
            mean_abs_error=0.0,
            error_std=0.0,
            temp_swing=0.0,
            pwm_mean=payload.current_state.pwm_output,
            pwm_max=payload.current_state.pwm_output,
            zero_crossings=0,
            in_band_ratio=0.0,
            overshoot_pct=0.0,
            settling_sec=None,
            saturation_ratio=0.0,
        )

    errors = [float(p.error) for p in points]
    temps = [float(p.current_temp) for p in points]
    targets = [float(p.target_temp) for p in points]
    pwms = [float(p.pwm_output) for p in points]
    ts_ms = [int(p.ts_ms) for p in points]

    in_band_ratio = sum(1 for err in errors if abs(err) <= payload.target_band) / len(errors)
    saturation_ratio = sum(1 for pwm in pwms if pwm >= payload.pwm_saturation_threshold) / len(pwms)

    overshoot_pct = 0.0
    for temp, target in zip(temps, targets):
        denom = max(abs(target), 1e-3)
        overshoot_pct = max(overshoot_pct, max(0.0, ((temp - target) / denom) * 100.0))

    return FeatureSet(
        mean_error=mean(errors),
        mean_abs_error=mean([abs(err) for err in errors]),
        error_std=pstdev(errors) if len(errors) > 1 else 0.0,
        temp_swing=max(temps) - min(temps),
        pwm_mean=mean(pwms),
        pwm_max=max(pwms),
        zero_crossings=_calc_zero_crossings(errors),
        in_band_ratio=in_band_ratio,
        overshoot_pct=overshoot_pct,
        settling_sec=_calc_settling_sec(ts_ms, errors, payload.target_band, payload.steady_window_samples),
```

Figure 6.1 – Feature extraction for control-behavior analysis

Table 6.1 – Main control-behavior indicators used for decision support

Indicator	Calculation basis	Engineering meaning
Mean absolute error	Absolute value of temperature error over the observation window	General tracking quality
In-band ratio	Share of samples inside the target band	Stability around the setpoint
Overshoot	Maximum temperature excess above target	Risk of excessive heating
Settling time	First time after which the error remains inside the target band	Speed of response
Temperature swing	Difference between maximum and minimum temperature in the window	Oscillation or instability
Saturation ratio	Share of samples at or above the PWM saturation threshold	Limited actuator headroom

6.3 Recommendation and feedback mechanism

The recommendation logic uses the extracted indicators to classify the observed behavior and select a limited parameter change. This is intentionally conservative. The mechanism should not produce large uncontrolled jumps in controller coefficients, because the physical temperature process has inertia and a delayed response. A recommendation must therefore be small enough to review and test through post-apply observation.

The classification step uses rule thresholds for behavior patterns. For example, frequent zero crossings together with high error spread indicate oscillation, high overshoot indicates an excessive transient response, and high saturation ratio indicates that the actuator has limited remaining authority. Slow response is detected when the mean absolute error remains high and the response does not settle within the expected time. These rules are simple, but they are transparent and can be checked against stored data.

The rule-based recommendation fragment is shown in Figure 6.2. It changes PID coefficients according to the detected problem type and limits the change step. For slow response it increases proportional and integral influence moderately. For steady-state error it mainly increases the integral term. For overshoot or oscillation it reduces

aggressive terms and can increase derivative damping. For saturation-limited behavior it treats the recommendation as higher risk. In all non-normal cases, explicit operator confirmation is required.

Source fragment: hmi/backend/app/services/ai/tuning_engine.py, lines 18–78

```python
def _derive_recommended(problem_type: ProblemType, current: PIDParams) -> tuple[PIDParams, ExpectedEffect]:
    kp, ki, kd = current.kp, current.ki, current.kd

    if problem_type == ProblemType.NORMAL:
        return _round_params(current), ExpectedEffect.KEEP_STABLE

    if problem_type == ProblemType.SLOW_RESPONSE:
        return _round_params(
            PIDParams(
                kp=kp + _bounded_step(kp, 0.12),
                ki=ki + _bounded_step(ki, 0.08),
                kd=max(0.0, kd - _bounded_step(kd, 0.06)),
            )
        ), ExpectedEffect.SPEED_UP_RESPONSE

    if problem_type == ProblemType.STEADY_STATE_ERROR:
        return _round_params(
            PIDParams(
                kp=kp,
                ki=ki + _bounded_step(ki, 0.15),
                kd=kd,
            )
        ), ExpectedEffect.REDUCE_STEADY_STATE_ERROR

    if problem_type == ProblemType.OVERSHOOT_HIGH:
        return _round_params(
            PIDParams(
                kp=max(0.0, kp - _bounded_step(kp, 0.1)),
                ki=max(0.0, ki - _bounded_step(ki, 0.08)),
                kd=kd + _bounded_step(kd, 0.12),
            )
        ), ExpectedEffect.REDUCE_OVERSHOOT

    if problem_type == ProblemType.OSCILLATION:
        return _round_params(
            PIDParams(
                kp=max(0.0, kp - _bounded_step(kp, 0.12)),
                ki=max(0.0, ki - _bounded_step(ki, 0.1)),
                kd=kd + _bounded_step(kd, 0.15),
            )
        ), ExpectedEffect.REDUCE_OSCILLATION

    return _round_params(
        PIDParams(
            kp=kp + _bounded_step(kp, 0.05),
            ki=ki,
            kd=kd,
        )
    ), ExpectedEffect.LIMITED_GAIN_EXPECTED


def build_recommendation(
    problem_type: ProblemType,
    current_params: PIDParams,
) -> tuple[PIDParams, PIDParams, PIDParams, RiskLevel, bool, ExpectedEffect]:
    recommended, effect = _derive_recommended(problem_type, current_params)
    delta = PIDParams(
        kp=round(recommended.kp - current_params.kp, 4),
        ki=round(recommended.ki - current_params.ki, 4),
        kd=round(recommended.kd - current_params.kd, 4),
    )
```

Figure 6.2 – Rule-based parameter recommendation fragment

The feedback mechanism closes the recommendation path. After a recommendation is applied through the HMI, the edge device must acknowledge the parameter update. Later telemetry is then compared with the pre-apply behavior and with

the predicted preview. This prevents the system from treating a recommendation as successful merely because a message was published.

6.4 Experimental setup

The validation was organized as a staged engineering check of the implemented closed-loop platform. The purpose was not to claim final industrial hardware certification, but to verify that the implemented layers work together: edge telemetry generation, MQTT exchange, Data Hub ingestion, storage, HMI monitoring, parameter command publication, device acknowledgement, decision-support preparation, and post-apply observation [2].

The main test environment used the ESP32-oriented edge firmware structure with the Wokwi/simulator profile, the Data Hub ingestion service, persistent storage, and the HMI frontend [11]. The simulator profile was used because the physical PCB and enclosure have been designed but not yet electrically and thermally tested as assembled hardware. This boundary is important: the validation confirms the software and system workflow, while final physical circuit validation must later be performed with instruments such as an oscilloscope, multimeter, external thermometer, and controlled load.

The validation scenarios are summarized in Table 6.2. They cover the main system behavior expected from a layered closed-loop temperature-control and monitoring system.

Table 6.2 – Validation scenarios for the developed system

Scenario	**Checked path**	**Expected result**
Telemetry ingestion	Edge telemetry → MQTT → Data Hub parsing → storage	Structured records are stored with device, setpoint, output, and status context
HMI monitoring	Stored telemetry → backend API → frontend screen	Current state and historical behavior are visible to the operator
Parameter update	HMI request → backend command path → MQTT params/set	Command is sent through the upper-layer command path
Acknowledgement	Edge validation → MQTT params/ack → Data Hub storage → HMI feedback	Operator can distinguish requested, rejected, staged, and applied commands

Scenario	Checked path	Expected result
Decision support	Stored history → feature extraction → recommendation generation	Recommendation is explainable and requires review
Post-apply verification	Applied parameters → later telemetry → before/after comparison	Actual behavior is compared with baseline and preview

6.5 Closed-loop tests

Closed-loop testing focused on whether the system preserves the engineering meaning of each action. A temperature sample should not disappear as an isolated screen value; it should become a stored telemetry fact. A parameter command should not be treated as successful until the device acknowledges it. A recommendation should not be treated as final until later behavior is observed.

The first group of tests verified telemetry continuity. The edge node generated periodic telemetry that included the target temperature, current process value, control error, controller output, PWM duty, saturation state, and controller parameters. The Data Hub ingested these records and stored them for later use by the HMI and decision-support mechanism. This confirmed the measurement-to-storage part of the loop.

The second group of tests verified parameter command handling. A new setpoint or controller parameter set was submitted through the HMI. The backend published the command through the configured MQTT path, and the edge device parsed and validated the payload. If the payload was accepted, the device returned an acknowledgement with the active runtime parameters. If the payload was invalid, the acknowledgement contained a failure result and a reason. This confirmed that the operator action was not reduced to a simple frontend event.

The third group of tests verified post-apply observation. After a parameter update, later telemetry was compared with the pre-apply baseline. The HMI validation view shown in Figure 6.3 presents the baseline, preview, and actual behavior around the application moment. This screenshot demonstrates the intended interpretation of decision support: the recommendation is useful only if the actual response after application can be observed and compared.

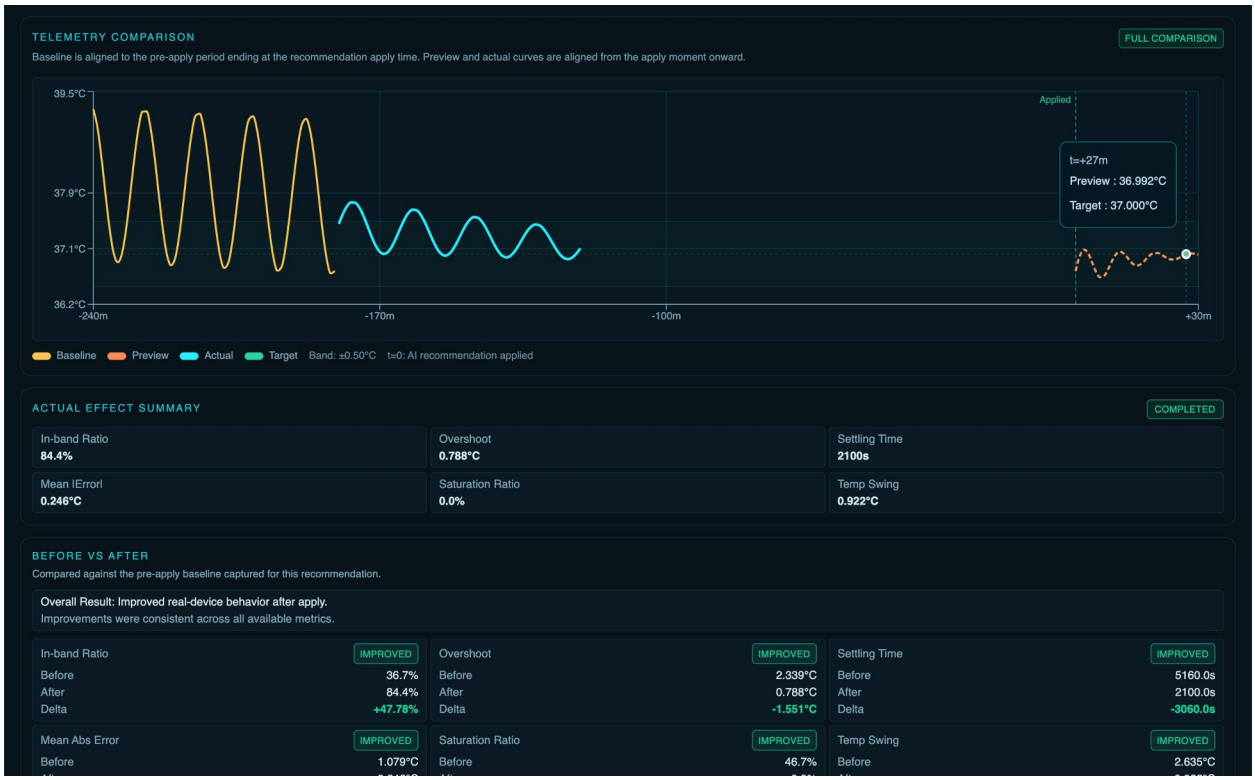

Figure 6.3 – HMI post-apply validation view with telemetry comparison

6.6 Telemetry and HMI validation

The HMI validation screen provides visual evidence of the post-apply verification workflow. In the shown validation case, the HMI compares the baseline behavior before the recommendation, the preview behavior prepared before application, and the actual behavior after application. The visualization also displays summary indicators such as in-band ratio, overshoot, settling time, mean absolute error, saturation ratio, and temperature swing.

The post-apply evaluator implementation is shown in Figure 6.4. It calculates actual metrics from observed telemetry points and compares them with a reference. The comparison is direction-aware: a higher in-band ratio is treated as improvement, while lower overshoot, lower mean absolute error, lower saturation ratio, lower temperature swing, and shorter settling time are treated as improvement. This makes the validation result more meaningful than a simple statement that the command was applied.

Source fragment: hmi/backend/app/services/ai/post_effect_evaluator.py, lines 27–93

```
    def calc_metrics(
        self,
        *,
        points: list[ObservedTelemetryPoint],
        target_band: float,
        pwm_saturation_threshold: float,
    ) -> Optional[PreviewMetrics]:
        if len(points) < 2:
            return None

        temps = [float(p.temp) for p in points]
        errors = [float(p.error) for p in points]
        abs_errors = [abs(v) for v in errors]

        in_band_ratio = sum(1 for v in abs_errors if v <= target_band) / len(abs_errors)
        overshoot_c = max(0.0, max(float(p.temp) - float(p.target_temp) for p in points))
        mean_abs_error = sum(abs_errors) / len(abs_errors)
        temp_swing = max(temps) - min(temps)

        sat_by_state = [str(p.saturation_state or "").strip().lower() for p in points]
        if any(v for v in sat_by_state):
            saturation_ratio = sum(1 for v in sat_by_state if v not in {"", "none", "normal"}) / len(points)
        else:
            saturation_ratio = sum(1 for p in points if float(p.pwm_output) >= pwm_saturation_threshold) /
len(points)

        settling_sec = self._calc_settling_sec(points=points, target_band=target_band)
        return PreviewMetrics(
            in_band_ratio=round(in_band_ratio, 6),
            overshoot_c=round(overshoot_c, 6),
            settling_sec=None if settling_sec is None else round(settling_sec, 6),
            mean_abs_error=round(mean_abs_error, 6),
            saturation_ratio=round(saturation_ratio, 6),
            temp_swing=round(temp_swing, 6),
        )

    def build_actual_summary(
        self,
        *,
        points: list[ObservedTelemetryPoint],
        metrics: PreviewMetrics,
    ) -> PostEffectMetrics:
        start_dt = datetime.utcfromtimestamp(points[0].ts_ms / 1000.0)
        end_dt = datetime.utcfromtimestamp(points[-1].ts_ms / 1000.0)
        return PostEffectMetrics(
            observed_window_start=start_dt,
            observed_window_end=end_dt,
            point_count=len(points),
            in_band_ratio_after=metrics.in_band_ratio,
            overshoot_c_after=metrics.overshoot_c,
            settling_sec_after=metrics.settling_sec,
            mean_abs_error_after=metrics.mean_abs_error,
            saturation_ratio_after=metrics.saturation_ratio,
            temp_swing_after=metrics.temp_swing,
        )

    def compare(self, *, reference: Optional[PreviewMetrics], actual: PreviewMetrics) -> PostEffectComparison:
        if reference is None:
            return PostEffectComparison()
        return PostEffectComparison(
            # Positive means improvement:
            # higher in-band is better, all other metrics lower is better.
            in_band_ratio_delta=round(actual.in_band_ratio - reference.in_band_ratio, 6),
            overshoot_c_delta=round(reference.overshoot_c - actual.overshoot_c, 6),
            settling_sec_delta=self._delta_optional(reference.settling_sec, actual.settling_sec),
            mean_abs_error_delta=round(reference.mean_abs_error - actual.mean_abs_error, 6),
            saturation_ratio_delta=round(reference.saturation_ratio - actual.saturation_ratio, 6),
            temp_swing_delta=round(reference.temp_swing - actual.temp_swing, 6),
```

Figure 6.4 – Post-apply effect evaluation fragment

The main result values visible in the HMI validation scenario are summarized in Table 6.3. These values are used as demonstration evidence for the software workflow

and should be interpreted as simulated or seeded validation data, not as final measurements of the untested physical PCB.

Table 6.3 – Example post-apply validation result from the HMI scenario

Indicator	Before application	After application	Result
In-band ratio	36.7 %	84.4 %	Improved
Overshoot	2.339 °C	0.788 °C	Improved
Settling time	5160 s	2100 s	Improved
Mean absolute error	1.079 °C	0.246 °C	Improved
Saturation ratio	46.7 %	0.0 %	Improved
Temperature swing	2.635 °C	0.922 °C	Improved

The result shows that the tested validation scenario successfully demonstrates the post-apply comparison path. The most important conclusion is not that the chosen parameters are universally optimal, but that the system can preserve enough information to evaluate whether a parameter change improved the observed behavior.

6.7 Results analysis

The validation confirms that the developed system implements the intended layered closed-loop workflow. The edge layer produces control-related telemetry and acknowledgements. The Data Hub parses and stores the records. The HMI provides monitoring, parameter configuration, command feedback, and post-apply observation. The auxiliary decision-support mechanism uses stored behavior to prepare reviewable recommendations and keeps the operator in the loop.

The experimental result also shows why persistent telemetry is necessary. Without stored baseline and post-apply windows, it would be impossible to distinguish a successful parameter change from a communication event that only appeared successful. The before/after metrics provide a clearer engineering basis for evaluation than the current temperature value alone.

At the same time, the result has clear limitations. The validation is based on the implemented software platform, simulator-oriented edge behavior, stored telemetry, and HMI demonstration data. The physical PCB, heater driver, sensor placement, and enclosure have not yet been fully verified under real electrical and thermal load. Therefore, the thesis should not claim final industrial readiness. The correct conclusion is that the developed prototype demonstrates a complete closed-loop engineering workflow and provides a foundation for later hardware deployment and instrument-based testing.

The chapter therefore completes the implementation evidence for the project. It connects decision support, HMI operation, command acknowledgement, and post-apply

behavior analysis into one traceable validation process. The final section of the thesis summarizes the obtained result and defines the remaining work required before a real hardware installation.

The most important engineering result of the validation is the preservation of causality across layers. A recommendation is generated from stored behavior, reviewed by the operator, applied through the command path, acknowledged by the edge device, and then evaluated against later telemetry. Each stage leaves a record that can be inspected independently. This makes the prototype stronger than a simple dashboard because it supports explanation of both successful and unsuccessful parameter changes.

The validation also defines the next experimental step. After the PCB is fabricated and assembled, the same metrics should be repeated with instrument-supported measurements. In that stage, oscilloscope traces of PWM switching, measured heater current, regulator stability, external temperature readings, and sensor-placement checks should be compared with the stored telemetry. The software workflow developed in this thesis is prepared for that later test because it already stores the data needed to compare command intent, device acknowledgement, and observed thermal response.

For this reason, the validation chapter should be read as a bridge between software verification and future physical testing. It proves that the implemented platform can collect the required evidence, calculate meaningful control-quality indicators, and present the result to the operator. The remaining hardware stage will reuse the same procedure with real sensor and actuator measurements, which means that the thesis result is not limited to a visual demonstration but forms a reproducible validation method for the later assembled node.

7 CONCLUSION

7.1 Achieved engineering results

The diploma project resulted in the design, implementation, and validation of a layered closed-loop temperature control and monitoring system. The developed system combines three main layers: the edge control layer, the Data Hub layer, and the HMI layer. It also includes an auxiliary decision-support mechanism, which analyzes stored behavior and prepares reviewable parameter recommendations. The decision-support part does not replace the operator and is not treated as a separate control layer.

The project confirmed the selected engineering direction. Local measurement, control calculation, safety checks, actuator-output preparation, telemetry publishing, command receiving, and acknowledgement generation are performed at the edge. This keeps the time-critical part of the control process close to the controlled object. The Data Hub receives MQTT messages, parses structured payloads, stores normalized records, tracks device status, and provides data for the HMI and later analysis. The HMI gives the operator access to monitoring, historical behavior, parameter configuration, command feedback, and post-apply observation.

The main engineering result is not only a temperature display or an isolated controller. The implemented prototype demonstrates a complete workflow: temperature measurement, local control, structured message exchange, Data Hub ingestion, persistent storage, HMI supervision, parameter update, device acknowledgement, and observation after the applied change. This satisfies the main aim of the work and provides a reproducible basis for further hardware-oriented development.

The edge layer was implemented with an ESP32-oriented firmware structure and a simulator profile. The firmware includes temperature acquisition, PI/PID-compatible control parameters, PWM output preparation, sensor-validity checks, fault handling, software safety forcing, MQTT telemetry, and acknowledgement handling. The hardware part of the project was also prepared at the design level. The schematic, PCB reference, component placement, Wokwi model, and enclosure design show how the firmware concept can be transferred toward a physical edge node. At the same time, the thesis deliberately does not claim final electrical or thermal validation of the assembled PCB. That stage must be completed later with a multimeter, oscilloscope or logic analyzer, external thermometer, controlled load, and recorded chamber-response tests.

The Data Hub and HMI layers were implemented as service components rather than as a single screen-oriented application. The Data Hub uses a bounded message-processing pipeline and supports persistent storage of telemetry, parameter commands, acknowledgements, summaries, alarm facts, and device status. The HMI backend and

frontend provide monitoring screens, historical views, parameter forms, acknowledgement-aware command feedback, and operations visibility. This structure makes operator action traceable and prevents a parameter update from being treated as successful only because a button was pressed or an MQTT message was published.

7.2 Deployment and validation support

The service deployment part of the project also supports the engineering result. Local infrastructure is prepared through Docker Compose files for PostgreSQL, TDengine, and Redis [10]. This makes the data and service environment reproducible during development, demonstration, and later testing. The repository also contains operational scripts for starting and stopping the HMI environment, resetting development databases, feeding telemetry, testing MQTT paths, and generating Data Hub load. The Data Hub stress script can publish telemetry from many simulated devices at a controlled message rate, which supports checking ingestion behavior, queueing, buffering, and saturation conditions. These tools are important because the developed system is a layered platform, and its quality depends not only on individual functions but also on service interaction under realistic operating conditions.

The deployment structure also makes the result easier to repeat during later thesis defense and laboratory work. Instead of manually preparing each service, the infrastructure can be started from documented compose files and helper scripts. The same environment can then be used for HMI demonstration, MQTT command testing, database reset, telemetry seeding, and Data Hub stress testing. This is important for an engineering thesis because the obtained result should be reproducible and inspectable, not dependent on a one-time manual launch sequence.

The auxiliary decision-support mechanism was implemented as a supporting analytical function. It extracts control-behavior indicators from stored history, classifies behavior such as slow response, overshoot, oscillation, steady-state error, or actuator saturation, and prepares parameter recommendations for operator review. The validation chapter demonstrated that recommendations can be connected to the same command path as manual changes and that later behavior can be compared with the baseline. This keeps the operator in the loop and supports explainability.

The validation results show that the developed prototype preserves the engineering meaning of the closed loop. Telemetry is not only displayed, but stored with context. Parameter changes are not only requested, but acknowledged by the edge device. Recommendations are not only generated, but can be checked through post-apply measurements. The example validation scenario showed improvement in the in-band ratio, overshoot, settling time, mean absolute error, saturation ratio, and temperature

swing. These values should be interpreted as software and simulation-based validation evidence, not as final physical hardware performance.

7.3 Limitations and further work

The main limitations of the current result are connected with the physical implementation stage. The PCB, power driver, enclosure, heater behavior, grounding, supply stability, and sensor placement still require real assembly and instrument-based testing. The simulator profile is useful for repeatable software validation, but it cannot prove MOSFET heating, PWM waveform quality, load-current behavior, real chamber dynamics, or sensor placement accuracy. Therefore, the next stage should include physical fabrication, electrical inspection, low-voltage power tests, PWM and gate waveform measurement, controlled load testing, external temperature comparison, and full chamber-response recording.

Further development can also include extended alarm handling, a stricter security model for operator roles, automatic report generation from validation runs, improved device provisioning, and comparison of several controller-parameter sets under the same test procedure. However, these improvements should be added above the current architecture without changing the main principle: control execution remains at the edge, message processing and storage remain in the Data Hub, operator supervision remains in the HMI, and decision support remains auxiliary.

The prepared validation method should also be reused during the first physical test campaign. The same command identifiers, acknowledgement records, parameter versions, and telemetry windows can connect instrument measurements with stored software records. For example, an oscilloscope trace of the PWM signal can be compared with the logged duty ratio, and an external thermometer reading can be compared with the stored sensor value. This will make the later hardware stage a continuation of the current engineering workflow rather than a separate undocumented experiment.

Thus, the objectives of the diploma project have been achieved. The work produced a layered closed-loop temperature control and monitoring system with edge execution, MQTT-based exchange, persistent storage, HMI operation, acknowledgement handling, auxiliary decision support, deployment support, stress-test tooling, and post-apply verification. The result is a complete engineering prototype and a prepared foundation for future real-hardware validation.

REFERENCES

[1] Åström K. J., Murray R. M. Feedback Systems: An Introduction for Scientists and Engineers. Princeton University Press, 2008.
[2] OASIS. MQTT Version 5.0. OASIS Standard. 07 March 2019. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html. Accessed: 14 May 2026.
[3] Espressif Systems. ESP32-WROOM-32 Datasheet. Version 3.6. Available: https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf. Accessed: 14 May 2026.
[4] Analog Devices. DS18B20 Programmable Resolution 1-Wire Digital Thermometer Data Sheet. Rev. 6. Available: https://www.analog.com/en/products/DS18B20.html. Accessed: 14 May 2026.
[5] Spring. Spring Boot Reference Documentation. Available: https://docs.spring.io/spring-boot/reference/index.html. Accessed: 14 May 2026.
[6] Project Reactor. Reactor Reference Guide. Available: https://projectreactor.io/docs/core/release/reference/. Accessed: 14 May 2026.
[7] TDengine. Get Started with TDengine TSDB Using Docker. Available: https://docs.tdengine.com/get-started/docker/. Accessed: 14 May 2026.
[8] FastAPI. FastAPI Documentation. Available: https://fastapi.tiangolo.com/. Accessed: 14 May 2026.
[9] React. React Reference Overview. Available: https://react.dev/reference/react. Accessed: 14 May 2026.
[10] Docker. Docker Compose Documentation. Available: https://docs.docker.com/compose/. Accessed: 14 May 2026.
[11] Wokwi. Wokwi Documentation. Available: https://docs.wokwi.com/. Accessed: 14 May 2026.
[12] PostgreSQL Global Development Group. PostgreSQL Documentation. Available: https://www.postgresql.org/docs/. Accessed: 14 May 2026.
[13] Åström K. J., Hägglund T. PID Controllers: Theory, Design, and Tuning. 2nd ed. Instrument Society of America, 1995.